JAVASCRIPT MASTERY SERIES · BOOK 1

JavaScript — Language & Syntax

The language itself, explained to be learned and kept: every construct and operator, with the common core taught in depth and the long tail mapped so nothing is invisible.

29 TOPICS · 8 PARTS · BUILT FOR LEARNING & PRINTING

How to read this

Start at the **Navigation Tree** — it maps the whole language; every name jumps to its section.

Each topic carries a tier badge. CORE topics are taught in depth, connected to real web & data-app work — learn these cold. REFERENCE topics are mapped lightly: enough to recognize and look up. Inside a topic, exhaustive lists (every operator, every loop form) are always shown so nothing stays hidden.

Code is syntax-highlighted. Look for When to use notes — they explain *why* you'd reach for a construct and which alternative to pick. Version notes flag when a feature landed (ES2020+). A **Syntax Index** at the end lists every operator and keyword.

Print or save as PDF with `Ctrl/Cmd + P` — the layout flows continuously and is print-tuned.

Navigation Tree

The whole language at a glance. teal = **core** (learn deeply) · grey = **reference** (know it exists). Every name links to its section.

- Parameters & arguments
- this & binding
- Closures
- Generators
- async / await

Objects & Classes

- Objects & object literals
- Destructuring
- Classes

Strings & Modules

- Template literals
- Modules — import · export

Iteration & Asynchrony

- Iterators & iterables
- Promises & the event loop

Part 1 · Fundamentals

How a program is spelled out: the statements you write, where your names live, and the one switch that makes the language stricter and safer.

Statements, expressions & comments CORE

> IN PLAIN TERMS
>
> A program is just a list of instructions you give the computer, carried out from top to bottom — like a recipe. Each instruction is called a **statement**. Some instructions *do* something ("save this", "repeat that"); others just *work out a value* ("2 plus 2", "this person's name") — those are called **expressions**. Knowing which is which sounds fussy, but it explains a lot of later rules. **Comments** are notes you leave for humans that the computer ignores — reminders to your future self about what a line is for.

A JavaScript program is a list of **statements** run top to bottom. A statement *does* something (declare a variable, loop, return); an **expression** *produces a value* (`2 + 2`, `user.name`, `getTotal()`). The distinction matters constantly: anywhere the language wants a value — a function argument, the right side of `=`, a template slot — you need an expression, not a statement.

Most statements end with a semicolon. JavaScript will insert missing ones for you (*Automatic Semicolon Insertion*), which works almost always but bites in a few spots — so the practical rule is: write the semicolons, or run a formatter that does.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`?.`	Optional chaining operator — safely reaches into nested data; if a piece of the path is missing it stops and gives `undefined` instead of crashing.
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`#`	Private field prefix — a `#` before a name inside a class marks it as truly private, reachable only from inside that class.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`argument`	Argument — the actual value you pass into a function when you call it.
`strict mode`	Strict mode — a stricter dialect that turns silent mistakes into clear errors; automatic in modules and classes.

Statements drive the program; expressions feed values into it.

```
// statements: each does something
let total = 0;            // declaration statement
total = price * quantity; // expression statement (assignment is itself an expression)
if (total > 100) ship();  // control-flow statement

// expressions: each produces a value
price * quantity          // arithmetic → a number
user?.name ?? "Guest"     // → a string
items.filter(inStock)     // → a new array
```

Comments

```
// single-line comment — to the end of the line
/* block comment —
   spans multiple lines */
/** JSDoc — tools read these for type hints & docs
 * @param {number} cents
 * @returns {string}
 */
function formatPrice(cents) { /* ... */ }
```

GOTCHA Automatic Semicolon Insertion mostly helps, but a line starting with `(`, `[`, `` ` ``, `,`, `+`, `-`, `, or` `/` can glue onto the previous line unexpectedly. A `return` followed by a newline is the classic trap — `return` on its own line returns `undefined` regardless of what's below it.

The building blocks

`statement` does something

An instruction: declaration, assignment, `if`, loop, `return`, etc. Statements don't nest where a value is expected.

`expression` produces a value

Anything that evaluates to a value. Function calls, arithmetic, literals, and `=` (assignment returns the assigned value) are all expressions.

`expression statement` `expr ;`

An expression used as a whole statement — usually a function call or assignment. `doThing();`

`block` `{ … }`

A group of statements treated as one unit, and (for `let`/`const`) a scope. Used by `if`, loops, functions.

`empty statement` `;`

A lone semicolon — does nothing. Occasionally used as an empty loop body; usually a typo.

`// comment`

Line comment. Everything after `//` to the end of the line is ignored.

`/ / comment`

Block comment. Cannot be nested.

`"use strict"` directive

A directive prologue (not really a statement) — see Strict mode.

`#!` hashbang

An executable script's first line (`#!/usr/bin/env node`) is allowed and ignored. **ES2023.**

Variables & scope CORE

> IN PLAIN TERMS
>
> A **variable** is a labeled box where you store a piece of information so you can use it again by name — like writing "price" on a box and putting a number inside. Instead of repeating the number everywhere, you just say "price." JavaScript gives you three words for making these boxes — `const`, `let`, and the old `var` — and they differ in two ways: whether you're allowed to put something new in the box later, and *where in your program the box can be seen* (that visibility is called **scope**). This section is about picking the right one, which is simpler than it sounds.

A variable is a name bound to a value. You'll declare them three ways — `const`, `let`, and (legacy) `var` — and the differences are about **scope** (where the name is visible) and **rebinding** (whether the name can point at something new).

The modern default is simple: **reach for `const` first, use `let` only when you must reassign, and avoid `var`**. This isn't style fussiness — `const`/`let` are block-scoped and predictable, while `var` leaks out of blocks and is silently hoisted, which is a real source of bugs in loops and conditionals.

▼ KEY TERMS IN THIS SECTION

`=`	Assignment operator — puts the value on its right into the variable on its left (it does *not* compare).
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`hoisting`	Hoisting — JavaScript's behaviour of noting declarations before running code, so some names exist before their line.
`temporal dead zone`	Temporal dead zone (TDZ) — the gap where a `let`/`const` name exists but can't be used yet; touching it early throws an error.
`binding`	Binding — the link between a name and the value it refers to.
`mutation`	Mutation — changing an existing object or array in place rather than making a new one.
`closure`	Closure — a function that remembers and keeps using variables from where it was created.
`module`	Module — a single code file with its own scope that shares code via `import`/`export`.

`const` freezes the binding, not the object. You can still mutate what it points to.

```
const API = "https://api.example.com";  // can't be reassigned
let page = 1;                           // will change as the user pages
page = page + 1;                        // fine

const user = { name: "Ada" };
user.name = "Grace";   // OK! const blocks REBINDING, not mutation
// user = {}           // TypeError: assignment to constant variable
```

Scope: where a name lives

`let` and `const` are **block-scoped** — they exist only inside the nearest `{ }`. `var` is **function-scoped** — it ignores blocks and is visible throughout the whole function, which surprises people inside `if` blocks and loops.

```
function demo(items) {
  if (items.length) {
    let n = items.length;   // visible only inside this if-block
    var v = items.length;   // visible in the WHOLE function
  }
  // console.log(n);  // ReferenceError — n is gone
  console.log(v);     // works — var leaked out
}
```

Hoisting & the temporal dead zone

Declarations are *hoisted* — the engine notes them before running the code. With `var`, the name exists from the top of its function as `undefined`. With `let`/`const`, the name is reserved but **off-limits until the declaration line** — touching it earlier *throws* (stops the program with an error). That guarded gap is the **temporal dead zone** (**TDZ**), and it turns silent `undefined` bugs into loud, easy-to-find errors.

```
console.log(a);  // undefined  (var is hoisted as undefined)
var a = 1;

console.log(b);  // ReferenceError: Cannot access 'b' before initialization
let b = 1;
```

GOTCHA The classic loop bug: `for (var i …) setTimeout(() => console.log(i))` logs the *final* `i` every time, because all callbacks share one function-scoped `i`. Switch to `let` and each iteration gets its own binding — the callbacks log `0,1,2…` as expected. This alone is a reason to drop `var`.

WHEN TO USE Older JavaScript had only `var`, which is scoped to the whole function and silently hoisted — so a variable declared inside an `if` or loop leaked out, and reused names overwrote each other in ways that caused subtle bugs (the loop-closure trap above is the famous one). `const` and `let` were introduced to fix that: they're **block-scoped**, so a name lives only where you'd expect, and the temporal dead zone turns "used too early" into a loud error instead of a silent `undefined`. `const` adds a second guarantee — the binding never changes — which lets a reader trust a value at a glance. **In practice:** default to `const` (most variables never need to be reassigned, and saying so documents intent), use `let` only when a value genuinely changes, and avoid `var` in new code — its only role now is reading legacy.

Declarations

`const` `const x = value`

Block-scoped, cannot be **reassigned** (the object it points to can still be mutated). Your default choice. Must be initialized at declaration.

`let` `let x = value`

Block-scoped, **reassignable**. Use when a value genuinely changes (counters, accumulators, current page). Can be declared without an initial value (`let x;` → `undefined`).

`var` `var x = value`

Legacy. Function-scoped, hoisted as `undefined`, allows redeclaration. Avoid in new code; you'll still read it in older codebases.

`Globals` `globalThis / window`

A variable declared outside any function at the top level of a *script* (not a module) becomes global. In modules, top-level names are module-scoped, not global — another reason modules are safer.

Strict mode REFERENCE

> IN PLAIN TERMS
>
> Early JavaScript was very forgiving — it would quietly let sloppy mistakes slide instead of complaining, which made bugs hard to find. **Strict mode** is a switch that tells JavaScript "be picky: stop me when I do something obviously wrong." You almost never turn it on by hand anymore, because modern code (anything using `import`/`export`, or `class`) runs in strict mode automatically. It's mostly here so you recognize it when you see it in older files.

Strict mode is a stricter dialect that catches silent mistakes and disables a few dangerous legacy behaviors. You almost never type it anymore: **ES modules and class bodies are strict automatically**, so most modern code already runs in it. It's worth recognizing in older files.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`export`	Export — marking something in a file as available for other files to import.
`import`	Import — bringing in something that another file has exported.

```
"use strict";          // must be the first line of a file or function

undeclared = 5;        // strict: ReferenceError (sloppy mode: creates a global!)

function f() {
  "use strict";        // strict for just this function
  // this === undefined here when called plainly (sloppy: this === globalThis)
}
```

What strict mode changes (highlights)

`Accidental globals` `throw`
Assigning to an undeclared name throws instead of silently creating a global.

`this in plain calls` `undefined`
A normal function call has `this === undefined` rather than the global object — surfacing `this` bugs.

`Duplicate params` `SyntaxError`
`function(a, a)` and a few other footguns become syntax errors.

`Read-only assignment` `throw`
Writing to non-writable properties throws instead of failing silently.

`Modules & classes` `always strict`
You don't opt in — `import`/`export` files and `class` bodies are strict by definition.

Part 2 · Values & Types

What data looks like in JavaScript, how values quietly turn into other types, and the comparison rules that decide whether two things count as “equal” or “truthy.”

The data types CORE

IN PLAIN TERMS

Every piece of information in a program has a **type** — a category that says what kind of thing it is and what you can do with it. A number, a piece of text, a yes/no value: these are different types, just like in real life you treat a phone number, a name, and a yes-or-no answer differently. JavaScript has eight of these categories. Most are simple single values (a number, some text); one of them, the **object**, is a container that holds many values together. Getting comfortable with these eight is the foundation for everything else.

JavaScript has exactly **eight types**: seven *primitives* (`string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`) and `object` (which includes arrays, functions, dates — everything else). Primitives are immutable, compared by value, and cheap; objects are mutable, compared by reference, and where all your structured data lives.

This split — value vs reference — is the single most important mental model for avoiding bugs. Copying a primitive copies the value; copying an object copies a *pointer* to the same thing, so two names can change one shared object.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`primitive`	Primitive — a basic single value (number, text, boolean, etc.) that is compared by its value and can't be changed in place.
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`immutable`	Immutable — cannot be changed after it's created; you make a new value instead of altering the old one.
`NaN`	NaN ("Not a Number") — the result of an invalid math operation, like `0/0`.
`iterator`	Iterator — the object that actually produces an iterable's items one by one via `.next()`.
`BigInt`	BigInt — a number type for integers too large for the normal number type to hold exactly.
`Symbol`	Symbol — a guaranteed-unique value, used mainly as a special, collision-proof object key.

Mutating through one reference is visible through every reference. This trips up state updates in UI code constantly.

```
// primitives — copied by value
let a = 10;
let b = a;      // b is its own 10
b++;            // a is still 10

// objects — shared by reference
const list1 = [1, 2, 3];
const list2 = list1;    // same array, two names
list2.push(4);          // list1 is now [1,2,3,4] too!
```

Checking a type

```
typeof "hi"         // "string"
typeof 42           // "number"
typeof 42n          // "bigint"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof Symbol()     // "symbol"
typeof {}           // "object"
typeof []           // "object"   (!) use Array.isArray([]) instead
typeof null         // "object"   (!) a famous historical bug
typeof function(){} // "function" (technically an object)
```

GOTCHA `typeof null === "object"` is a 25-year-old bug that can't be fixed without breaking the web. To test for `null`, compare directly: `value === null`. To test arrays, use `Array.isArray(value)`, never `typeof`.

WHEN TO USE Two of these types exist to express *absence*, and people mix them up — so it's worth knowing why both are there. `undefined` is the language's own "nothing here yet": it's what you get from an uninitialized variable, a missing property, or a function with no `return`. `null` exists so *you* can say "intentionally empty" — a deliberate signal that's distinct from "never set." Keeping them separate lets code tell "the server hasn't answered" apart from "the server answered, and the value is nothing." `bigint` was added later for a concrete problem: `number` is a 64-bit float and loses precision past ~9 quadrillion, so large database IDs and currency-in-cents totals would silently corrupt — `bigint` holds them exactly. **In practice:** let the language produce `undefined`; reach for `null` to mean deliberate-empty; use `bigint` only when values exceed the safe-integer range, since it can't mix with `number`.

The eight types

`string` `text`
Immutable UTF-16 text: `"hi"`, `'hi'`, `` `hi` ``. Every "change" makes a new string. Full method surface lives in the String built-in (Book 2).

`number` `64-bit float`
All ordinary numbers — there is no separate integer type. Holds integers exactly up to `Number.MAX_SAFE_INTEGER` (2^53–1); beyond that, use `bigint`. Includes `NaN` and `Infinity`.

`bigint` `big integers`
Arbitrary-precision integers, written with an `n` suffix: `9007199254740993n`. For IDs and money math that exceed safe-integer range. Can't be mixed with `number` in arithmetic. **ES2020.**

`boolean` `true / false`
Logical truth. The result of comparisons and the input to conditionals.

`undefined` `absent`
The default "no value yet" — uninitialized variables, missing properties, missing arguments, and functions with no `return`.

`null` `intentional empty`
An *explicit* "nothing here," set by you to signal emptiness. Convention: `undefined` = not set; `null` = deliberately set to nothing.

`symbol` `unique key`
A guaranteed-unique value, mainly used as non-colliding object keys and to hook into language protocols (`Symbol.iterator`). **ES2015.**

`object` `everything else`

Keyed collections of values. Arrays, functions, dates, maps, DOM nodes, plain `{}` — all objects. Mutable and compared by reference.

Type conversion & coercion CORE

IN PLAIN TERMS

> Sometimes information is the wrong *type* for what you're doing — the most common example: anything a user types into a web form arrives as **text**, even when it's a number. The text `"42"` is not the number `42`, and adding it to something can go wrong. **Conversion** is when you deliberately change one type into another ("turn this text into a real number"). **Coercion** is when JavaScript does it *automatically* behind your back — which is convenient but causes famous surprises. This section is about doing it on purpose so the surprises never happen to you.

Values change type in two ways: **conversion** you ask for (`Number(x)`, `String(x)`), and **coercion** the engine does automatically when an operator meets mismatched types. Coercion is where JavaScript earns its reputation — `"5" * 2` is `10` but `"5" + 2` is `"52"` — so the goal is to understand it well enough to convert *explicitly* and never rely on the surprises.

Real-world trigger: data from forms, URLs, and JSON arrives as **strings**. `"42"` from an `<input>` is text; add it to a number and you'll concatenate, not sum. Convert at the boundary.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`%`	Remainder (modulo) operator — gives what's left over after division (e.g. `7 % 3` is `1`); handy for wrapping and even/odd checks.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`operator`	Operator — a symbol that performs an action on values, like `+` (add) or `===` (compare).
`NaN`	NaN ("Not a Number") — the result of an invalid math operation, like `0/0`.
`concatenation`	Concatenation — joining pieces of text end to end into one string.
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.
`radix`	Radix — the number base used when reading text as a number; `10` means ordinary decimal.

Convert explicitly (recommended)

The form-input fix: `const qty = Number(input.value)` *before doing math.*

```
Number("42")         // 42       (NaN if it can't parse the whole string)
parseInt("42px", 10) // 42       (reads leading digits; always pass the radix 10)
parseFloat("3.14em") // 3.14
String(42)           // "42"
(42).toString()      // "42"
Boolean(0)           // false
JSON.parse(text)     // text → structured data (API responses)
JSON.stringify(obj)  // data → text (sending to a server / localStorage)
```

Coercion the engine does for you

The `+` *operator is the troublemaker: if either side is a string, it concatenates.*

```
"5" + 2      // "52"   + with any string ⇒ concatenation
"5" - 2      // 3      - * / % have no string meaning ⇒ numeric coercion
5 + true     // 6      true → 1, false → 0
5 + null     // 5      null → 0
5 + undefined // NaN   undefined → NaN
[] + {}      // "[object Object]"  (objects → strings via toString)
`Total: ${total}`    // template literals coerce to string
```

GOTCHA `Number("")` is `0` and `Number("  ")` is `0` — empty/whitespace strings become zero, which can hide a missing form value. If you need "empty means invalid," check the string before converting.

WHEN TO USE JavaScript was designed to be forgiving — it auto-converts types so a script never crashes on a mismatch — but that convenience is exactly what produces `"5" + 2 === "52"` and `[] == false`. You can't remove coercion from the language, so the winning strategy is to make it *irrelevant* by converting on purpose. **Why convert at the edges:** data arrives as text from forms, URLs, and JSON, so its type is wrong before you ever compute with it; if you fix the type the moment it enters (`Number(input.value)`, `JSON.parse(body)`), every line downstream can trust it and you never debug a coercion surprise deep in your logic. **In practice:** coerce explicitly at the boundary where data enters, then rely on the types everywhere after — and treat any reliance on implicit `+` / `==` coercion as a smell.

Conversion toolkit

`Number(x)` → `number`
Strict numeric conversion; whole string must be numeric or you get `NaN`. Best for validated input.

`parseInt(s, 10)` → `integer`
Parses a leading integer, ignoring trailing junk (`"42px"` → `42`). **Always pass the radix** `10` to avoid legacy octal surprises.

`parseFloat(s)` → `number`
Like `parseInt` but keeps decimals.

`String(x)` → `string`
Safe stringify of anything, including `null` / `undefined`. `x.toString()` throws on those.

`Boolean(x)` → `boolean`
Applies truthiness rules (next topic). `!!x` is the terse idiom.

`Unary +` `+x` → `number`
Shorthand numeric conversion: `+"42"` → `42`. Readable in moderation.

`JSON.parse / stringify` `text ⟷ data`
The boundary between text (network, storage) and live objects. Covered fully under JSON in Book 2.

Equality & truthiness CORE

IN PLAIN TERMS

Two everyday questions in any program: "are these two things the same?" and "is this thing basically a yes?" JavaScript has a clean way to answer both — but also some old, confusing ways you should avoid. For comparing, there's a right tool (`===`) and a trap (`==`). For yes/no, every value counts as either "truthy" (treated as yes) or "falsy" (treated as no), and there's a short list of the falsy ones worth memorizing. Get these two ideas down and your `if` statements will behave exactly as you expect.

Two questions you'll ask in nearly every conditional: *are these two values equal?* and *is this value "on"?* JavaScript gives you a clean answer to both if you follow two rules: **use `===` (never `==`), and know the seven falsy values by heart.**

`===` (strict equality) compares without coercion — same type and same value. `==` (loose equality) coerces first, producing genuinely confusing results (`0 == ""` , `null == undefined` , `[] == false` are all `true`). There is no good reason to use `==` in modern code, with one tiny exception noted below.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`!==`	Strict inequality operator — the opposite of `===` ; `true` when two values are *not* strictly equal.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===` .
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`||`	Logical OR operator — `true` if either side is truthy; also returns the first truthy value, which powers fallback shortcuts.
`&&`	Logical AND operator — `true` only if both sides are truthy; also used to run something only when a condition holds.
`?:`	Conditional (ternary) operator — a one-line if/else that produces a value: `condition ? valueIfTrue : valueIfFalse` .
`0n`	BigInt literal — a number written with an `n` suffix (like `10n`) to represent very large integers exactly.
`coercion`	Coercion — JavaScript *automatically* converting a value from one type to another (e.g. number to text).
`NaN`	NaN ("Not a Number") — the result of an invalid math operation, like `0/0` .
`nullish`	Nullish — meaning specifically `null` or `undefined` (i.e. "missing"), as opposed to other falsy values.

Objects compare by reference: `{} === {}` is `false` — two different objects, even if identical in content.

```
0 === "0"      // false  (number vs string — no coercion)
0 == "0"       // true   (== coerces — avoid)
null === undefined  // false
null == undefined   // true  (the one handy ==: checks "null or undefined")

NaN === NaN    // false  (!) NaN equals nothing, even itself
Number.isNaN(x)       // the correct NaN test
Object.is(NaN, NaN)   // true  — like === but NaN-aware and -0-aware
```

Truthiness — the falsy seven

In any boolean context (`if`, `&&`, `||`, `?:`, `Boolean()`), every value is either *truthy* or *falsy*. Memorize the **falsy** list — everything else is truthy:

An empty array `[]` is truthy — to check for "no items," test `arr.length`, not the array itself.

```
// the ONLY falsy values:
false, 0, -0, 0n, "", null, undefined, NaN

// therefore these are all TRUTHY (a common surprise):
"0", "false", [], {}, function(){}   // non-empty string, empty array/object — all truthy!

if (items.length) render(items);   // idiom: 0 is falsy, so this means "if not empty"
const name = input.value || "Guest";   // fallback when value is falsy
```

GOTCHA `||` falls back on *any* falsy value, so `count || 10` replaces a legitimate `0` with `10`. When `0` or `""` are valid, use `??` (nullish coalescing) instead — it only falls back on `null`/`undefined`. See the next part.

Equality operators

`===` strict equal

Same type and value, no coercion. **Your default.**

`!==` strict not-equal

Negation of `===`.

`==` loose equal

Coerces operands before comparing. Avoid — except the idiom `x == null` to mean "is null or undefined."

`!=` loose not-equal

Coercing inequality. Avoid for the same reasons.

`Object.is(a, b)` → boolean

Like `===` but treats `NaN` as equal to `NaN` and distinguishes `+0` from `-0`. Niche but correct for those edge cases.

`Number.isNaN(x)` → boolean

The right way to detect `NaN` (since `NaN === NaN` is false).

Part 3 · Operators

The symbols that combine values. Listed exhaustively so none stays mysterious — with the everyday ones

taught in depth and the rare ones mapped.

Arithmetic & assignment operators CORE

IN PLAIN TERMS

Operators are the little symbols that combine or change values — you already know most of them from math class: `+`, `-`, `*`, `/`. **Arithmetic** operators do the calculating. **Assignment** operators put a result into a variable (the `=` sign, plus handy shortcuts like `+=` that mean "add this to what's already there"). This is the everyday math of programming — counting things, totaling a cart, stepping through items.

Arithmetic operators do the math; assignment operators write results back into variables. The two combine into the compact `+=` / `-=` family you'll use in loops, accumulators, and running totals.

Two everyday traps: `+` means *concatenation* the moment a string is involved (`"$" + total`), and all `number` math is floating-point, so `0.1 + 0.2` is `0.30000000000000004`. For money, work in integer cents or use a decimal library.

▼ KEY TERMS IN THIS SECTION

`=`	Assignment operator — puts the value on its right into the variable on its left (it does *not* compare).
`!==`	Strict inequality operator — the opposite of `===`; `true` when two values are *not* strictly equal.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`??=`	Nullish assignment operator — assigns a value only if the variable is currently `null` or `undefined`; used to set defaults.
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`||`	Logical OR operator — `true` if either side is truthy; also returns the first truthy value, which powers fallback shortcuts.
`&&`	Logical AND operator — `true` only if both sides are truthy; also used to run something only when a condition holds.
`>>>`	Zero-fill right shift operator — shifts bits right, filling with zeros (always gives a positive result).
`<<`	Left shift operator — shifts a number's bits left (a fast multiply by powers of two).
`>>`	Right shift operator — shifts a number's bits right, keeping its sign.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`--`	Decrement operator — subtracts 1 from a number variable (`i--`).
`%`	Remainder (modulo) operator — gives what's left over after division (e.g. `7 % 3` is `1`); handy for wrapping and even/odd checks.
`#`	Private field prefix — a `#` before a name inside a class marks it as truly private, reachable only from inside that class.
`operand`	Operand — a value that an operator works on; in `a + b`, both `a` and `b` are operands.
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`coercion`	Coercion — JavaScript *automatically* converting a value from one type to another (e.g. number to text).
`truthy`	Truthy — a value treated as "yes/true" in a condition (almost everything).
`falsy`	Falsy — a value treated as "no/false" in a condition; the short list is `false, 0, "", null, undefined, NaN`.
`NaN`	NaN ("Not a Number") — the result of an invalid math operation, like `0/0`.
`concatenation`	Concatenation — joining pieces of text end to end into one string.

`nullish` Nullish — meaning specifically `null` or `undefined` (i.e. "missing"), as opposed to other falsy values.

`%` *(remainder) is the workhorse for wrapping indexes, alternating rows, and pagination math.*

```
let total = 0;
for (const item of cart) total += item.price;    // running sum

const avg = sum / count;
const remainder = index % columns;    // % = remainder — great for grid wrapping
const area = side ** 2;               // ** = exponentiation (ES2016)

let i = 0;
i++;      // post-increment: returns old value, then adds
++i;      // pre-increment: adds, then returns new value
```

GOTCHA `0.1 + 0.2 !== 0.3`. This is IEEE-754 floating point, not a JS quirk. For currency, store integer cents (`1099` = $10.99) and format on display, or use a dedicated decimal type.

Arithmetic

`+` `a + b`

Addition — **or string concatenation** if either operand is a string. The overloading is the #1 coercion surprise.

`-` `a - b`

Subtraction. Always numeric (coerces strings to numbers).

`*` `a * b`

Multiplication.

`/` `a / b`

Division. `1/0` is `Infinity`, `0/0` is `NaN` — never a thrown error.

`%` `a % b`

Remainder (not true modulo — keeps the sign of the dividend). Used for wrapping and even/odd tests.

`**` `a ** b`

Exponentiation: `2 10` → `1024`. **Right-associative.** ES2016.**

`++` `x++ / ++x`

Increment by 1. Prefix returns the new value, postfix the old.

`--` `x-- / --x`

Decrement by 1.

`unary -` `-x`

Negation; also coerces to number.

`unary +` `+x`

Coerces to number without changing sign: `+"42"` → `42`.

Assignment

`=` `x = v`

Assign. Is itself an expression returning `v`, which is why `a = b = 0` works.

+= -= *= /= %= `x += v`

Compound arithmetic-assign: `x += 1` is `x = x + 1`. `+=` also concatenates strings.

****=** `x **= v`

Exponentiate-assign. **ES2016.**

&&= ||= ??= `x ??= v`

Logical assignment: assign **only if** the left side is truthy (`&&=`), falsy (`||=`), or nullish (`??=`). `opts.timeout ??= 3000` sets a default without clobbering a real `0`. **ES2021.**

<<= >>= >>>= &= |= ^= `x &= v`

Compound bitwise-assign — see Bitwise operators.

Comparison & logical operators CORE

IN PLAIN TERMS

These are the operators that let your program make decisions. **Comparison** operators ask true/false questions: is this bigger, smaller, equal? **Logical** operators combine those questions: "is the user logged in AND an admin?", "is it a weekend OR a holiday?" The results feed into your `if` statements and control what happens next. There's also a neat trick here — these operators can hand back a useful value, not just true/false — which powers some very common shortcuts you'll see everywhere.

Comparisons produce booleans; logical operators combine them — and, crucially, `&&` and `||` don't just return `true`/`false`, they **return one of the operands**. That's what powers the `cond && <render>` and `value || fallback` idioms you see everywhere in UI code.

Both `&&` and `||` **short-circuit**: they stop evaluating as soon as the result is known. `user && user.name` never touches `.name` when `user` is null — a guard pattern that predates optional chaining.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`!==`	Strict inequality operator — the opposite of `===`; `true` when two values are *not* strictly equal.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`||`	Logical OR operator — `true` if either side is truthy; also returns the first truthy value, which powers fallback shortcuts.
`&&`	Logical AND operator — `true` only if both sides are truthy; also used to run something only when a condition holds.
`operand`	Operand — a value that an operator works on; in `a + b`, both `a` and `b` are operands.
`coercion`	Coercion — JavaScript *automatically* converting a value from one type to another (e.g. number to text).
`truthy`	Truthy — a value treated as "yes/true" in a condition (almost everything).
`falsy`	Falsy — a value treated as "no/false" in a condition; the short list is `false, 0, "", null, undefined, NaN`.
`optional chaining`	Optional chaining — using `?.` to read nested data without crashing when part of it is missing.
`short-circuit`	Short-circuiting — when `&&`/`||` stop early because the result is already decided, skipping the rest.

`a || b` *yields* `a` *if truthy, else* `b`. `a && b` *yields* `a` *if falsy, else* `b`. *They return values, not just booleans.*

```
age ⩾ 18 && hasConsent      // both must hold
isAdmin || isOwner          // either suffices
!isLoading                  // negate

// short-circuit returns an operand, not a boolean:
const name = input.value || "Guest";   // first truthy (or last value)
const port = config.port ?? 3000;      // ?? for null/undefined only (next topic)
loggedIn && showDashboard();           // run RHS only if LHS truthy
list.length > 0 && render(list);       // conditional render idiom
```

GOTCHA Comparing with `<`, `>` etc. coerces strings to numbers — but comparing two *strings* sorts lexicographically: `"10" < "9"` is `true` (character order). Convert to `Number` before comparing numeric strings.

Comparison

`===` `!==` strict (in)equality

No coercion — your default. (Full treatment under Equality & truthiness.)

`==` `!=` loose (in)equality

Coerces first. Avoid except `x == null`.

`>` `<` greater / less than

Numeric for numbers; lexicographic for two strings.

`⩾` `⩽` greater/less or equal

As above, inclusive.

Logical

`&&` `a && b`

Logical AND. Returns `a` if falsy, else `b`. Short-circuits. Used for conditional execution/rendering.

`||` `a || b`

Logical OR. Returns `a` if truthy, else `b`. Short-circuits. Used for fallbacks — but beware falsy `0`/`""` (use `??`).

`!` `!a`

Logical NOT — coerces to boolean and flips it. `!!x` converts any value to its boolean.

Nullish coalescing & optional chaining CORE

IN PLAIN TERMS

Real data is often *missing* — a user hasn't filled in their middle name, a server didn't send a field, a setting was never chosen. These two tools (added in 2020) handle missing data gracefully. **Nullish coalescing** (`??`) means "use this value, but if it's missing, fall back to that one instead." **Optional chaining** (`?.`) means "try to reach this nested piece of data, but don't crash if part of the path isn't there." Together they replace a lot of ugly, error-prone safety checks with two tiny symbols.

These two ES2020 operators are the modern way to deal with possibly-missing data — exactly the situation you're in with API responses, optional config, and nested objects. They've quietly replaced piles of defensive `&&` chains and `||` fallbacks.

`??` is the precise fallback: it substitutes only when the left side is `null` or `undefined`, leaving legitimate `0`, `""`, and `false` intact. `?.` reaches into nested structures and stops safely (yielding `undefined`) the moment it hits `null`/`undefined`, instead of crashing the program with the error "cannot read property of undefined."

▼ KEY TERMS IN THIS SECTION

`?.`	Optional chaining operator — safely reaches into nested data; if a piece of the path is missing it stops and gives `undefined` instead of crashing.
`??=`	Nullish assignment operator — assigns a value only if the variable is currently `null` or `undefined`; used to set defaults.
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`||`	Logical OR operator — `true` if either side is truthy; also returns the first truthy value, which powers fallback shortcuts.
`&&`	Logical AND operator — `true` only if both sides are truthy; also used to run something only when a condition holds.
`falsy`	Falsy — a value treated as "no/false" in a condition; the short list is `false, 0, "", null, undefined, NaN`.
`rest`	Rest — using `...` to collect several values into a single array or object.

The everyday pattern for API data: `?.` to descend safely, `??` to supply a default at the end.

```
// ?? — fallback only for null/undefined (unlike ||)
const pageSize = settings.pageSize ?? 25;   // keeps a real 0
const label    = product.name ?? "Untitled";

// ?. — safe navigation through maybe-missing links
const city = user?.address?.city;           // undefined if user or address is missing
const first = response?.data?.items?.[0];    // safe index access
user?.profile?.save?.();                     // call only if it exists

// combine them: reach safely, then default
const theme = user?.prefs?.theme ?? "light";
```

GOTCHA You can't mix `??` with `||`/`&&` without parentheses — `a ?? b || c` is a SyntaxError on purpose, to prevent ambiguity. Wrap the intended grouping: `(a ?? b) || c`.

VERSION Both `??` and `?.` are **ES2020**. `&&=`/`||=`/`??=` (their assignment forms) are **ES2021**. Safe in every current browser and Node ≥14.

WHEN TO USE Before 2020 the fallback idiom was `x || default`, but `||` triggers on *any* falsy value — so a real `0`, `""`, or `false` would get silently replaced by the default, a genuinely common bug in forms and settings. `??` exists to draw the

line in the right place: fall back **only** on `null` / `undefined`, leaving legitimate falsy values intact. `?.` solves a parallel pain: reaching into data you don't control (API responses, optional config) used to require defensive `a && a.b && a.b.c` chains, and forgetting one link threw "cannot read property of undefined" and crashed the page. Optional chaining makes the whole path fail *safely* to `undefined`. **Why people reached for them instantly:** together they replaced piles of defensive boilerplate with two characters each. **In practice:** use `??` when `0` / `""` / `false` are valid; use `?.` on uncertain/external data — not on values you know exist, where it just masks bugs.

The operators

`??` `a ?? b`

Nullish coalescing — returns `b` only when `a` is `null` or `undefined`. The correct fallback when `0` / `""` / `false` are valid values.

`?.` `a?.b`

Optional chaining (property). Yields `undefined` instead of throwing if `a` is `null` / `undefined`. Short-circuits the rest of the chain.

`?.[ ]` `a?.[key]`

Optional chaining for dynamic/computed keys and array indexes.

`?.( )` `fn?.(args)`

Optional call — invoke only if the function exists. Great for optional callbacks: `onChange?.(value)`.

`??=` `a ??= b`

Assign `b` only if `a` is nullish. Sets defaults in place. **ES2021.**

Bitwise operators REFERENCE

IN PLAIN TERMS

Deep down, computers store everything as 1s and 0s (**bits**). **Bitwise** operators let you flip and combine those individual bits directly. In normal web and data work you'll almost never need them — they show up in specialized corners like graphics, encryption, or compact on/off permission flags. They're included here so the picture is complete and you recognize them, not because you'll reach for them often.

Bitwise operators treat numbers as 32-bit binary and manipulate individual bits. In typical web and data-app work you'll rarely need them — they show up in flags/permissions bitmasks, color/byte manipulation, hashing, and graphics. Listed here for completeness and recognition.

▼ KEY TERMS IN THIS SECTION

- `!==` Strict inequality operator — the opposite of `===`; `true` when two values are *not* strictly equal.
- `==` Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
- `>>>` Zero-fill right shift operator — shifts bits right, filling with zeros (always gives a positive result).
- `<<` Left shift operator — shifts a number's bits left (a fast multiply by powers of two).
- `>>` Right shift operator — shifts a number's bits right, keeping its sign.
- `&` Bitwise AND operator — combines two numbers bit by bit; used for low-level flag/mask work.
- `|` Bitwise OR operator — combines two numbers bit by bit; used to set flags.
- `^` Bitwise XOR operator — sets each bit where exactly one of the two inputs has it.
- `~` Bitwise NOT operator — flips every bit of a number.

The one common real use: combining and testing bit-flag permissions.

```
const READ = 1, WRITE = 2, EXEC = 4;    // permission flags
let perms = READ | WRITE;               // combine → 3
const canWrite = (perms & WRITE) !== 0; // test a flag → true
perms &= ~WRITE;                        // clear a flag
```

GOTCHA Bitwise ops coerce operands to **32-bit signed integers**, so they silently truncate large numbers and decimals. People sometimes abuse `|0` or `~~x` to floor a number — readable code prefers `Math.trunc` / `Math.floor`.

Bit operations

& `a & b`

AND — bit set only where both are set. Used to *test/mask* flags.

| `a | b`

OR — bit set where either is set. Used to *combine* flags.

^ `a ^ b`

XOR — bit set where exactly one is set. Toggling, simple parity.

~ `~a`

NOT — inverts every bit (`~n` equals `-(n+1)`).

<< `a << n`

Left shift — multiply by 2^n (drops high bits).

>> `a >> n`

Sign-propagating right shift — integer divide by 2^n, keeping sign.

>>> `a >>> n`

Zero-fill right shift — shifts in zeros; result is always unsigned.

Unary & relational operators CORE

IN PLAIN TERMS

This is a small toolbox of operators for *inspecting* values rather than calculating with them. The stars are three plain-English questions you'll ask all the time about data you receive: `typeof` ("what kind of thing is this?"), `instanceof` ("is this one of those?"), and `in` ("does this object have that property?"). They're especially useful when you're handed data from elsewhere and need to check what you actually got before using it.

A grab-bag of single-operand and membership operators. The headliners are `typeof` (which type is this?), `instanceof` (is this an X?), and `in` (does this object have this key?) — all three are everyday tools for inspecting unknown data, especially untyped JSON from the network.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`?:`	Conditional (ternary) operator — a one-line if/else that produces a value: `condition ? valueIfTrue : valueIfFalse`.
`operator`	Operator — a symbol that performs an action on values, like `+` (add) or `===` (compare).
`operand`	Operand — a value that an operator works on; in `a + b`, both `a` and `b` are operands.
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`coercion`	Coercion — JavaScript *automatically* converting a value from one type to another (e.g. number to text).
`ternary`	Ternary — the `? :` operator; a compact if/else that yields a value.
`prototype`	Prototype — the object a JavaScript object inherits shared properties and methods from.
`instance`	Instance — a specific object built from a class blueprint.
`constructor`	Constructor — the special method that sets up a new instance when you use `new`.
`generator`	Generator — a special function that can pause and resume, producing values on demand.
`promise`	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.

`in` checks for a key's existence — distinct from checking its value, which matters when `undefined` / `0` / `false` are legitimate stored values.

```
typeof value === "string"        // type guard before using string methods
"email" in formData              // does the key exist? (true even if value is undefined)
err instanceof TypeError         // which kind of error did we catch?
arr instanceof Array             // (but prefer Array.isArray(arr))

delete cache[staleKey];          // remove a property from an object
void 0                           // produces undefined (rarely needed)
```

GOTCHA `instanceof` breaks across execution contexts (e.g. arrays from another iframe/realm) and follows the prototype chain. For arrays specifically, `Array.isArray()` is the reliable check; for plain type tags, `typeof` or `Object.prototype.toString` are steadier.

Unary operators

`typeof` `typeof x`

Returns a type-name string (`"string"`, `"number"`, `"object"`, `"function"`, …). Safe on undeclared names — handy for feature detection.

delete `delete obj.k`

Removes a property from an object. Returns `true` / `false`. Doesn't work on variables; not for arrays (leaves a hole — use `splice`).

void `void expr`

Evaluates `expr` and returns `undefined`. Mostly seen as `void 0` or in bookmarklets.

! `!x`

Logical NOT (also under Logical). `!!x` → boolean coercion.

- + `-x  +x`

Numeric negation / numeric coercion.

~ `~x`

Bitwise NOT (see Bitwise).

await `await p`

Unary operator that suspends an `async` function until a promise settles (full treatment under async/await).

Relational & membership

in `key in obj`

`true` if the key exists on the object or its prototype chain. Tests existence, not value.

instanceof `x instanceof C`

`true` if `C.prototype` is in `x`'s prototype chain — "is x an instance of C?" Used in `catch` blocks to branch on error type.

Other operators worth knowing

?: `cond ? a : b`

The conditional (ternary) operator — the only one taking three operands. An *expression* form of if/else (see Conditionals).

new `new C(args)`

Constructs an instance from a class/constructor function (see Classes).

new.target `meta`

Inside a function, tells whether it was called with `new`. Niche.

`, a, b`

Comma operator — evaluates both, returns the last. Rare outside `for` headers; usually avoid for clarity.

yield yield* `in generators`

Pause/produce a value (and delegate) inside a generator function (see Generators).

Spread & rest CORE

IN PLAIN TERMS

Both of these are written as three dots `...`, and they do opposite, very handy jobs. **Spread** *unpacks* a collection — think of emptying a bag onto a table so you can see and copy all the items at once (great for copying a list, or merging two together). **Rest** does the reverse — it *gathers* loose items back into one bag (great for a function that accepts "any number of things"). Once you see the three dots, just ask: is it pouring out, or scooping up?

Same three dots `...`, two opposite jobs. **Spread** *expands* an iterable (anything you can step through, like an array or string) or object into its pieces — copying arrays, merging objects, passing array elements as arguments. **Rest** *gathers* leftover pieces into one array or object — collecting extra function arguments, or "everything else" when unpacking a value (covered under Destructuring).

Spread is the idiomatic way to make **shallow copies** and immutable updates, which is exactly what state-driven UIs (React, etc.) need: never mutate the old value, build a new one.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`immutable`	Immutable — cannot be changed after it's created; you make a new value instead of altering the old one.
`mutate`	Mutate — to change a value in place (the opposite of immutable).
`shallow`	Shallow copy — a copy one level deep; nested objects inside are still shared with the original, not duplicated.
`destructuring`	Destructuring — unpacking values from an object or array into separate variables in one line.
`iterable`	Iterable — any value you can step through one item at a time with `for…of` (lists, text, and more).

Spread — expand

*Object spread (`{...obj}`) is **ES2018**; array spread is **ES2015**. Both copies are shallow — nested objects are still shared.*

```
const copy   = [...items];                 // shallow array copy
const merged = [...a, ...b];               // concatenate
const next   = [...list, newItem];         // immutable append
const obj2   = { ...user, name: "Grace" }; // copy + override a field
Math.max(...numbers);                      // array → individual args
greet(...["Hi", "Ada"]);                   // spread into parameters
```

Rest — gather

*Rest must be the **last** parameter or binding. It always produces a genuine array (unlike the old `arguments`).*

```
function sum(...nums) {            // collect all args into a real array
  return nums.reduce((a, n) ⇒ a + n, 0);
}
const [first, ...others] = ranking;       // first item, rest as array
const { id, ...fields } = record;         // pull id out, keep the remainder
```

GOTCHA Spread copies are **shallow**: `const c = {...state}` shares any nested objects/arrays with the original, so mutating `c.list.push(x)` still affects `state`. For deep copies use `structuredClone(value)` (modern) when you truly need independence.

WHEN TO USE These exist to make **copying and recombining** data a one-liner — and, more importantly, to make *immutable* updates ergonomic. The problem they solve: objects and arrays are shared by reference, so mutating one in place can secretly change it everywhere it's referenced, which is the root of countless state bugs. Spread lets you build a *new* value from an old one (`{...state, count: 1}`) instead of mutating, so the original stays untouched — this is precisely why React/Redux-style code is built on it: predictable updates depend on never changing the previous state. Rest solves the inverse: before it, collecting "all the arguments" meant the clunky array-like `arguments` object (absent in arrow functions); `...args` gives you a real array directly, and `{a, ...rest}` cleanly expresses "take this, keep the remainder." **In practice:** spread for immutable copy-with-changes and to pass arrays as arguments; rest for flexible signatures and partial destructuring. The copy is shallow — use `structuredClone` for depth.

The two roles of ...

spread (array) `[...iterable]`

Expands any iterable into elements — copy, merge, append, convert a Set/NodeList to an array. **ES2015.**

spread (object) `{...obj}`

Copies own enumerable properties; later keys win on conflict. The immutable-update idiom. **ES2018.**

spread (call) `fn(...args)`

Passes array elements as individual arguments — replaces `apply`.

rest (params) `function(...args)`

Collects all/extra arguments into one array. The modern replacement for `arguments`.

rest (array) `[a, ...rest]`

In destructuring, captures remaining elements as an array. Must be last.

rest (object) `{a, ...rest}`

Captures remaining own properties as a new object. Must be last. Great for "omit a field" patterns. **ES2018.**

Part 4 · Control Flow

Choosing what runs and how often: branching, repetition, and handling the things that go wrong along the way.

Conditionals: if, else, switch CORE

> IN PLAIN TERMS
>
> **Conditionals** are how a program chooses between paths — the "if this, then that" logic at the heart of every app. `if` / `else` is the everyday version: "if the user is logged in, show the dashboard; otherwise, show the login page." `switch` is a tidier way to handle one value with many possible cases (like a vending machine reacting to which button was pressed). And there's a compact one-line form (the **ternary**) for simple either/or choices. This is decision-making, plain and simple.

Branching is most of programming. `if` / `else if` / `else` is the general tool; the ternary `?:` is its compact *expression* form (it produces a value, so it slots into assignments and template literals); `switch` shines when one value is matched against many fixed cases.

A practical style note that prevents bugs: keep branches small and prefer **early returns** (guard clauses) over deep nesting. Handle the edge cases up front, return, and let the happy path read straight down the function.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`?:`	Conditional (ternary) operator — a one-line if/else that produces a value: `condition ? valueIfTrue : valueIfFalse`.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`statement`	Statement — a single complete instruction in a program (one step of the recipe).
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`truthy`	Truthy — a value treated as "yes/true" in a condition (almost everything).
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.

The ternary `cond ? a : b` is the only conditional that returns a value — reach for it in expression positions, not as a statement-replacement for multi-line logic.

```
// if / else if / else
if (status === "loading") showSpinner();
else if (status === "error") showError();
else render(data);

// early-return guard clauses — flatter, clearer
function getDiscount(user) {
  if (!user) return 0;              // bail early
  if (!user.isMember) return 0;
  return user.years >= 5 ? 0.2 : 0.1;   // ternary as an expression
}

// ternary inside a template (expression position)
const msg = `You have ${count} ${count === 1 ? "item" : "items"}`;
```

switch

`switch` compares with `===`. Each `case` falls through to the next unless you `break` or `return` — intentional fall-through (stacked cases) is the one good use of that behavior.

```
switch (action.type) {
  case "increment":
    return state + 1;          // return exits — no break needed
  case "decrement":
    return state - 1;
  case "reset":
  case "clear":                // fall-through: shared handling
    return 0;
  default:
    return state;              // always handle the unknown case
}
```

GOTCHA Forgetting `break` is the classic `switch` bug: execution "falls through" into the next case and runs it too. Either `return` from each case (common in reducers) or end each with `break`. Also: to declare `let`/`const` inside a case, wrap that case body in `{ }` to give it its own scope.

WHEN TO USE `switch` and the ternary exist not because `if` can't do their jobs, but because *expressing intent* matters. A long `else if` ladder comparing the same variable to many constants hides the fact that it's really "match one value against a list" — `switch` makes that shape obvious and lets cases share handling via fall-through. The ternary exists because `if` is a *statement* (it does something) while sometimes you need a *value* in place — inside an assignment, a template literal, or a JSX attribute — and `cond ? a : b` is an *expression* that produces one. **Why people use each:** clarity at a glance. **In practice:** `if`/`else if` for general or range conditions; `switch` when matching one value against many fixed cases (action types, statuses); the ternary only in value positions, never as a stand-in for multi-step logic — nested ternaries are where readability dies.

Conditional forms

`if (…) …` `statement`
Runs its block when the condition is truthy. The general-purpose branch.

`else / else if` `statement`
Alternative branches. Chain `else if` for ordered tests; the first truthy one wins.

?: `cond ? a : b`

Ternary — the expression form of if/else. Returns a value; nest sparingly (it gets unreadable fast).

switch / case `statement`

Matches one value against many `case` labels with `===`. Use `break` / `return` per case; stack cases for shared handling.

default `in switch`

The fallback case when nothing matches. Always include one.

Loops CORE

IN PLAIN TERMS

A **loop** is how you repeat something without writing it out over and over — "do this for every item in the list." If you have a hundred products to display, you don't write a hundred lines; you write one loop that runs a hundred times. JavaScript has a few flavors of loop for slightly different situations, plus some newer "methods" that loop *for* you in a cleaner way. The goal of this section is helping you pick the one that says what you mean most clearly.

JavaScript gives you several loops, and picking the right one makes code read like its intent. The modern default for walking values is `for…of` (arrays, strings, Maps, Sets, anything iterable). Use the classic `for` when you need the index or custom stepping, and `while` when you loop until a condition rather than over a collection.

For transforming data you'll often skip explicit loops entirely in favor of array methods — `map`, `filter`, `reduce`, `forEach` (Book 2). They're more declarative and compose well. Reach for a real loop when you need to `break` early, `await` inside, or do genuinely imperative work.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`mutation`	Mutation — changing an existing object or array in place rather than making a new one.
`iterable`	Iterable — any value you can step through one item at a time with `for…of` (lists, text, and more).
`generator`	Generator — a special function that can pause and resume, producing values on demand.
`promise`	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.

*`for…of` gives you the **values**; `for…in` gives you the **keys** — mixing them up is a common bug.*

```
// for...of — the everyday value loop
for (const product of cart) {
  console.log(product.name);
}

// classic for — when you need the index or step
for (let i = 0; i < rows.length; i++) {
  rows[i].position = i;
}

// for...of with entries() to get index + value
for (const [i, item] of items.entries()) { /* ... */ }

// while / do...while
while (queue.length) process(queue.shift());
do { tick(); } while (running);   // body runs at least once
```

for…in vs for…of

Avoid `for…in` on arrays — it yields string indexes and can pick up inherited keys. Use `for…of` for arrays, `Object.keys/entries` for objects.

```
const arr = ["a", "b", "c"];
for (const v of arr) { }   // "a", "b", "c"   ← values ✅ for arrays
for (const k in arr) { }   // "0", "1", "2"   ← keys (strings!) ⚠️

// for...in is for plain OBJECT keys — but prefer Object.keys:
for (const key of Object.keys(config)) { /* ... */ }

// awaiting in sequence (only works in for...of, not .forEach):
for (const url of urls) {
  const res = await fetch(url);   // runs one at a time
}
```

GOTCHA `await` inside `.forEach()` does **not** wait — `forEach` ignores the promises (the "I'll finish later" objects async work returns; see *Promises*) and races ahead. To await sequentially, use a `for…of` loop; to run in parallel, use `await Promise.all(items.map(...))`.

WHEN TO USE JavaScript accumulated several loops over time, and the newer forms exist to remove old foot-guns. The classic `for (let i…)` works but forces you to manage an index you often don't care about, and `for…in` (meant for object keys) misbehaves on arrays. `for…of` was introduced to iterate *values* of anything iterable directly and safely, while still supporting `break` and `await`. But the deeper shift is toward **array methods** (`map` / `filter` / `reduce`): they exist because most loops are really *transformations*, and a loop buries that intent in mechanics while mutating as it goes. `map` says "produce a new array from this one" in a single readable expression, returns a fresh array (no mutation), and composes with other methods. **In practice:** `for…of` to walk values, classic `for` when you truly need the index, array methods to transform data — and a real loop only when you must `break` early or `await` each step in sequence.

Loop forms

`for…of` `for (const x of iterable)`

Iterates **values** of any iterable (array, string, Map, Set, NodeList, generator). Supports `break` / `continue` / `await`. Your default.

for `for (init; cond; step)`

Classic counted loop. Use when you need the index, a custom step, or to iterate backwards.

for…in `for (const k in obj)`

Iterates enumerable **keys** (including inherited). For plain objects only — and even then `Object.keys()` is usually clearer. Not for arrays.

while `while (cond) …`

Repeats while the condition holds; may run zero times. For "loop until done" logic.

do…while `do … while (cond)`

Like `while` but the body always runs **at least once** (condition checked after).

forEach `arr.forEach(fn)`

Array method, not a loop keyword — runs `fn` per element. Can't `break` and doesn't await. (Full array methods in Book 2.)

---

break, continue & labels REFERENCE

IN PLAIN TERMS

Sometimes, partway through a loop, you want to bail out or skip ahead. `break` means "stop looping entirely, I'm done" (you found what you were searching for). `continue` means "skip the rest of this round and move on to the next item" (this one doesn't qualify, next!). **Labels** are a rarely-needed way to aim those commands at an outer loop when you have loops nested inside loops. Two of these you'll use often; the third you'll mostly just recognize.

`break` exits a loop early; `continue` skips to the next iteration. Both are everyday tools inside real loops. **Labels** — names you attach to a loop so `break` / `continue` can target an *outer* loop from inside a nested one — are rare; most code is clearer if you extract the nested loop into a function and `return` instead.

▼ KEY TERMS IN THIS SECTION

`===` Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.

`==` Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.

`rest` Rest — using `...` to collect several values into a single array or object.

Without the label, `break` would only exit the inner loop. Labels reach outward — but extracting a function and returning is usually cleaner.

```
for (const row of rows) {
  if (row.hidden) continue;      // skip this one
  if (row.id === target) break; // found it — stop entirely
  process(row);
}

// labeled break — escape both loops at once (rare)
outer:
for (const row of grid) {
  for (const cell of row) {
    if (cell === needle) break outer;   // exits BOTH loops
  }
}
```

Jump statements

`break` `break;`

Exits the nearest enclosing loop or `switch` immediately.

`continue` `continue;`

Skips the rest of the current iteration and starts the next.

`label:` `name: loop`

Names a loop so `break` / `continue` can target it from a nested loop. Niche; prefer refactoring to a function.

`break label` `break name;`

Breaks out of the labeled (outer) loop.

`continue label` `continue name;`

Continues the labeled (outer) loop's next iteration.

Error handling CORE

IN PLAIN TERMS

Things go wrong in real programs — the network drops, a user types nonsense, a file is missing. **Error handling** is how you deal with that gracefully instead of letting the whole app crash. The idea: "**try** this risky thing; if it fails, **catch** the problem and respond calmly (show a friendly message, retry)". You can also deliberately raise an alarm yourself (**throw**) when something isn't right. This is what separates an app that shows "Something went wrong, please retry" from one that simply freezes.

Things fail — network calls, bad input, missing data. `try` / `catch` / `finally` lets you attempt risky work, respond to failure, and clean up regardless. You `throw` to signal a problem; you `catch` to handle it. Done well, error handling keeps a UI responsive (show a message, retry) instead of crashing.

Throw `Error` objects, never bare strings — `Error` carries a `message`, a `name`, and a `stack` trace that's invaluable in debugging. Subclasses (`TypeError`, `RangeError`) and your own `class AppError extends Error` let `catch` blocks branch on *what* went wrong.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`binding`	Binding — the link between a name and the value it refers to.
`constructor`	Constructor — the special method that sets up a new instance when you use `new`.
`rejected`	Rejected — a promise that has failed and now holds an error.

`finally` *always runs — perfect for teardown like hiding a spinner or closing a connection.*

```
try {
  const res = await fetch("/api/orders");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  // fetch doesn't throw on 404!
  const data = await res.json();
  render(data);
} catch (err) {
  console.error(err);
  showToast("Couldn't load orders — please retry.");
} finally {
  hideSpinner();   // runs whether we succeeded or failed
}
```

Throwing and custom errors

Branch on error type with `instanceof`*, and* ***re-throw*** *anything you didn't mean to swallow.*

```
function withdraw(balance, amount) {
  if (amount ⩽ 0) throw new RangeError("Amount must be positive");
  if (amount > balance) throw new Error("Insufficient funds");
  return balance - amount;
}

class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;          // attach context for the caller
  }
}

try { /* ... */ }
catch (err) {
  if (err instanceof ValidationError) highlightField(err.field);
  else throw err;                // re-throw what you can't handle
}
```

GOTCHA `fetch()` only rejects on *network* failure — an HTTP 404 or 500 still resolves successfully. You must check `res.ok` and throw yourself, as above. Also: a `return` inside `try` still runs `finally` before actually returning.

VERSION Optional catch binding — `catch {` with no `(err)` — is **ES2019**, for when you don't need the error object. `Error` `cause` (`new Error(msg, { cause })`) for chaining is **ES2022**.

WHEN TO USE `throw` / `try` / `catch` exist to separate "what should happen" from "what to do when it can't" — without them, every function would have to return error codes and every caller would have to check them, drowning the real logic in plumbing. Throwing lets a deep function abandon ship and a distant caller decide how to recover, with the failure traveling up automatically. Throwing `Error` **objects** (not strings) matters because they carry a `message`, a `name`, and a `stack` trace — the breadcrumb trail that makes debugging possible. **Custom** `Error` **subclasses** exist so callers can branch on *what* went wrong (`catch (e) { if (e instanceof ValidationError) … }`) instead of fragile string-matching on messages, and can attach structured context (which field failed). **In practice:** throw when a function genuinely can't fulfill its contract; return a value for expected non-error outcomes; subclass `Error` when callers need to distinguish failure types.

Error-handling constructs

`try { }` `block`

Wraps code that might throw. Must be followed by `catch` and/or `finally`.

`catch (err) { }` `block`

Runs if anything in `try` throws; `err` is the thrown value. The binding is optional (ES2019) when unused.

`finally { }` `block`

Always runs — after success, after a caught error, even after a `return`. For cleanup.

`throw` `throw value`

Raises an error, unwinding to the nearest `catch`. Throw `Error` objects, not strings.

`Error` `new Error(msg)`

The base error type — carries `message`, `name`, `stack`. Subclass it for domain-specific errors.

`TypeError / RangeError / …` `built-in`

Standard error subclasses thrown by the language; also useful to throw yourself for precise meaning.

`async rejection` `.catch() / try`

Rejected promises are caught by `try/catch` around `await`, or `.catch()` on the chain — an unhandled rejection is a top-level error.

Part 5 · Functions

The unit of reuse and the heart of JavaScript’s expressiveness — how to define them, feed them, bind their <code>this</code>, capture state in closures, and run them asynchronously.

Defining functions CORE

> IN PLAIN TERMS
>
> A **function** is a reusable bundle of instructions with a name — you write the steps once, then "call" it by name whenever you need them, instead of repeating yourself. Think of it like a kitchen appliance: you put ingredients in (the *inputs*), it does its job, and hands something back (the *result*). Functions are the main way programs are organized. JavaScript lets you write them in a few different shapes, and this section walks through each shape and when you'd use it.

There are three shapes you'll use daily: the **declaration** (`function name() {}`), the **expression** (a function assigned to a variable), and the **arrow** (`() => {}`). They mostly do the same job; the differences that matter are *hoisting* and how they treat `this` .

Practical default: use **arrow functions for callbacks** (`.map` , event handlers, `setTimeout`) because they don't rebind `this` , and use **declarations for named top-level functions** because they hoist and read clearly. Arrows also have a tidy *implicit return* for one-liners.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name` .
`declaration`	Declaration — the act of creating a named thing (a variable or function) for the first time.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`hoisting`	Hoisting — JavaScript's behaviour of noting declarations before running code, so some names exist before their line.
`TDZ`	TDZ (temporal dead zone) — the period before a `let` / `const` is initialized, where using it throws an error.
`binding`	Binding — the link between a name and the value it refers to.
`lexical`	Lexical — "based on where it's written in the source code" (as opposed to how it's run).
`callback`	Callback — a function you hand to another function to be run later (e.g. when a click happens).
`generator`	Generator — a special function that can pause and resume, producing values on demand.
`promise`	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.

Arrow one-liners implicitly return their expression. To return an object literal, wrap it in parentheses so `{}` isn't read as a function body.

```
// declaration — hoisted (callable before its line)
function area(w, h) { return w * h; }

// expression — assigned to a const, not hoisted
const area2 = function (w, h) { return w * h; };

// arrow — concise; inherits `this` from its surroundings
const area3 = (w, h) => w * h;          // implicit return (no braces)
const double = n => n * 2;              // one param: parens optional
const make = () => ({ ready: true });   // return an object: wrap in ()

// arrows shine as callbacks
const names = users.map(u => u.name);
button.addEventListener("click", () => save());
```

Declaration vs expression: hoisting

Function declarations are fully hoisted; function expressions and arrows follow the variable rules (TDZ for `const`/`let`).

```
greet();                      // ✅ works — declarations are hoisted
function greet() { return "hi"; }

greet2();                     // ❌ ReferenceError (TDZ) / TypeError
const greet2 = () => "hi";
```

GOTCHA Arrow functions have **no own** `this`, **no** `arguments`, **and can't be** `new`**-ed**. That's a feature for callbacks but a problem for object methods that need their own `this` (use a regular method there) — covered next in *this and binding*.

WHEN TO USE The three forms exist because they answer different needs, and the differences that matter are **hoisting** and `this`. The **declaration** is the original: it's hoisted (usable before its line) and shows a clean name in stack traces — ideal for the named, reusable functions that form a program's skeleton. The **function expression** came from treating functions as values you can store and pass; it isn't hoisted, which is sometimes exactly what you want. The **arrow** was added to solve a specific, painful bug: ordinary functions get their *own* `this` based on how they're called, so a callback passed to `.map` or `setTimeout` would lose the surrounding `this` and break — developers worked around it with `var self = this` or `.bind(this)` everywhere. Arrows fix it at the language level by **capturing** `this` **from where they're written**, which is why they instantly became the default for callbacks. **In practice:** declaration for named top-level functions; arrow for callbacks and anything passed around; expression for a function-as-value without hoisting; and *not* an arrow for an object/class method that needs its own `this` (use method shorthand there).

Ways to define a function

`declaration` `function f() {}`
Named, **hoisted**, has its own `this`/`arguments`. Best for standalone named functions.

`expression` `const f = function(){}`
A function as a value. Not hoisted. Can be named for better stack traces (`function inner(){}`).

`arrow` `(a) => expr`
Concise; **lexical** `this` (inherits from enclosing scope); implicit return for single expressions. Ideal for callbacks. Not for methods/constructors.

`method shorthand` `{ run() {} }`
Inside object/class literals: `name() {}` defines a method without the `function` keyword.

generator `function* g() {}`

Produces values lazily via `yield` (see Generators).

async `async function f() {}`

Returns a promise; enables `await` inside (see async/await).

IIFE `(function(){})()`

Immediately-invoked function expression — runs once to create a private scope. Largely replaced by modules and blocks.

Function() `new Function(...)`

Builds a function from strings at runtime. Rare; a security/perf hazard like `eval`. Avoid.

Parameters & arguments CORE

IN PLAIN TERMS

Functions become powerful when you can feed them different information each time you use them. **Parameters** are the labeled slots a function defines for its inputs ("this function expects a *price* and a *quantity*"). **Arguments** are the actual values you hand over when you call it ("give it 9.99 and 3"). Same idea as a form with blank fields (parameters) that you fill in (arguments). This section covers how to set up those slots flexibly — with sensible defaults and optional values — so your functions are pleasant to use.

Parameters are the names in the definition; arguments are the values you actually pass. JavaScript is relaxed here — extra arguments are ignored, missing ones become `undefined` — so modern features (**defaults**, **rest**, **destructured params**) exist to make that flexibility safe and readable.

The most useful real-world pattern is the **options object**: instead of many positional parameters you can't remember the order of, accept a single `{ }` and *destructure* it — that is, pull the named values out of it (a technique covered fully in its own section) — with defaults. It self-documents at the call site and makes arguments order-independent and optional.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`||`	Logical OR operator — `true` if either side is truthy; also returns the first truthy value, which powers fallback shortcuts.
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`rest`	Rest — using `...` to collect several values into a single array or object.
`destructure`	Destructure — to pull values out of an object or array into named variables in one step.

The `= {}` after the destructure lets the whole argument be omitted, so calling `createUser()` with nothing won't crash with an error.

```
// default parameters — used when the arg is undefined
function paginate(items, page = 1, size = 25) {
  const start = (page - 1) * size;
  return items.slice(start, start + size);
}
paginate(rows);            // page=1, size=25
paginate(rows, 2);         // page=2, size=25

// rest parameter — gather the rest into an array (must be last)
function log(level, ...messages) {
  console[level](...messages);
}

// destructured options object — the readable way to pass many options
function createUser({ name, role = "member", active = true } = {}) {
  return { name, role, active };
}
createUser({ name: "Ada", role: "admin" });   // order-free, self-documenting
```

arguments (legacy) vs rest

Prefer rest parameters: `arguments` isn't a real array, doesn't exist in arrow functions, and reads worse.

```
// old: the array-like `arguments` object (not in arrow functions)
function oldSum() {
  return Array.from(arguments).reduce((a, n) => a + n, 0);
}
// modern: rest gives a real array directly
const sum = (...nums) => nums.reduce((a, n) => a + n, 0);
```

GOTCHA Defaults trigger only on `undefined`, **not** `null`: `f(null)` keeps `null`, it does not fall back to the default. And passing an object/array argument passes a *reference* — mutating it inside the function mutates the caller's value.

WHEN TO USE The **options-object** pattern exists to fix a real readability failure: `createUser(true, false, 3)` is meaningless at the call site — you can't tell what the booleans mean or remember the order, and adding a new option means touching every caller. Passing a single destructured object (`createUser({ role: "admin", active: true })`) makes each argument self-labeling, order-independent, and individually optional, so you can add options later without breaking existing calls. **Default parameters** exist so "if not provided, use this" lives in the signature instead of as `x = x || fallback` lines in the body (which also wrongly triggered on `0`). **Why people adopted it:** APIs that survive change. **In practice:** switch to an options object once you have more than two or three arguments, any optional ones, or any booleans; keep positional params for one or two clearly-ordered required values.

Parameter features

positional `function f(a, b)`
Matched by order. Missing trailing args are `undefined`; extra args are ignored.

default `function f(a = 1)`
Used when the argument is `undefined`. Can reference earlier params and call functions. **ES2015.**

rest `function f(...args)`
Collects remaining args into a real array. Must be last. Replaces `arguments`.

destructured `function f({a, b})`

Unpacks an object/array argument into named locals — the basis of the options-object pattern. Add `= {}` to make it optional.

arguments `arguments object`

Legacy array-like of all args, available in non-arrow functions only. Prefer rest.

trailing comma `f(a, b,)`

Allowed in params and calls — keeps diffs clean. **ES2017.**

---

this and binding CORE

IN PLAIN TERMS

The word `this` is JavaScript's way of saying "the thing I'm currently working with." Picture telling several people "clean *your* room" — who *your* refers to depends on who you're talking to. `this` is the same: the very same function can mean different things by `this` depending on *how it gets called*, not where it was written. That flexibility is handy but trips up nearly everyone at first, because a function can lose track of what `this` was meant to be. This section makes the rule clear and shows how to keep `this` pointed where you want.

`this` is the most misunderstood word in JavaScript, but the rule is learnable: **`this` is set by *how a function is called*, not where it's defined** — except arrow functions, which capture `this` from their surrounding scope and never change it. Master that one distinction and the confusion mostly evaporates.

Where this bites in real apps: you pass an object method as a callback (`setTimeout(obj.method)`, an event handler) and `this` is no longer the object. The fixes are arrow functions, `.bind()`, or class fields — all shown below.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`scope`	Scope — the region of code where a particular variable is visible and usable.
`binding`	Binding — the link between a name and the value it refers to.
`spread`	Spread — using `...` to expand a list or object into its individual pieces.
`callback`	Callback — a function you hand to another function to be run later (e.g. when a click happens).
`instance`	Instance — a specific object built from a class blueprint.
`module`	Module — a single code file with its own scope that shares code via `import` / `export`.
`strict mode`	Strict mode — a stricter dialect that turns silent mistakes into clear errors; automatic in modules and classes.

*The four call patterns set `this` differently: **method** call (`obj.fn()`) → `obj`; **plain** call (`fn()`) → `undefined` in strict mode; `new` → the new instance; **explicit** (`call`/`apply`/`bind`) → whatever you pass.*

```
const counter = {
  count: 0,
  // method call: `this` is the object to the left of the dot
  increment() { this.count++; },
};
counter.increment();      // this === counter ✅

// detached call loses `this`:
const inc = counter.increment;
inc();                    // this is undefined (strict) → TypeError ❌

// fixes:
setTimeout(() => counter.increment(), 100);   // arrow keeps outer `this` ✅
const bound = counter.increment.bind(counter);// permanently bound ✅
```

Arrows capture this lexically

Inside `start()`, the arrow's `this` is whatever `start()`'s `this` was — the instance. This is the reason arrows dominate callbacks.

```
class Timer {
  seconds = 0;
  start() {
    // arrow: `this` is the Timer instance, inherited from start()
    setInterval(() => { this.seconds++; }, 1000);   // ✅
    // a regular function here would have its own (wrong) `this` ❌
  }
  // class field + arrow = a method permanently bound to the instance:
  onClick = () => { this.seconds = 0; };
}
```

GOTCHA Don't use an arrow function as an **object method** if it needs the object as `this` — an arrow captures the *outer* `this` (often the module or `undefined`), not the object. Use method shorthand (`obj = { run() {} }`) for methods; use arrows for callbacks *inside* them.

WHEN TO USE `this` is dynamic in JavaScript — it's set by *how* a function is called, not where it's defined — which is powerful but means a method detached from its object (passed as a callback) silently loses its `this` and breaks. The binding tools all exist to pin `this` back down. The **arrow function** is the modern answer: it has no `this` of its own and captures the surrounding one, so a callback "just works" — this is why it replaced the old `var self = this` and scattered `.bind` calls. The **class field arrow** (`onClick = () => {}`) exists so a method you'll hand to an event listener stays bound to its instance permanently. `.bind()` predates arrows and is still useful for pinning `this` (and presetting arguments) on a named function you can't rewrite. **In practice:** arrow for inline callbacks; class-field arrow for handlers you pass around; `.bind()` to adapt existing functions; a real method (not an arrow) when the function *should* receive its object as `this`.

Controlling this

`method call` `obj.fn()`

`this` is `obj` (the value before the dot). The everyday case.

`plain call` `fn()`

`this` is `undefined` (strict mode / modules) or the global object (sloppy). A frequent source of bugs when methods get detached.

`new call` `new Fn()`

`this` is the brand-new instance being constructed.

`arrow` `() ⇒ …`

No own `this` — captures it from the enclosing scope at definition time. Immune to how it's called.

`.bind(thisArg)` `fn.bind(obj)`

Returns a new function permanently bound to `obj` (and optionally preset leading args). Used to fix detached methods.

`.call(thisArg, …args)` `fn.call(obj, a, b)`

Invokes immediately with a chosen `this` and individual args.

`.apply(thisArg, args[])` `fn.apply(obj, [a, b])`

Like `call` but takes an args array. Spread (`fn(...args)`) has largely replaced it.

`class field arrow` `onX = () ⇒ {}`

Defines a per-instance bound method — the clean fix for passing handlers as callbacks. **ES2022.**

Closures CORE

> IN PLAIN TERMS
>
> Imagine a chef who trained at a special restaurant. Even after they leave and cook elsewhere, they still carry that restaurant's secret recipes in their head. A **closure** is the same idea for code: when you create a function inside another function, the inner one *remembers* the surrounding variables it was born with, and can keep using them later — even after the outer function has finished. Why care? Because it lets a function keep its own private information that nothing else can peek at or change. It sounds abstract, but you're already using closures every time you write a button click handler — this section just names the thing you've been doing.

A closure is simply this: **a function remembers the variables from where it was created, even after that outer function has finished.** Every callback, event handler, and module already relies on closures — you've been using them without naming them.

They're the foundation for *private state* (data only a returned function can touch), *factory functions* (build customized functions on demand), and *memoization/caching*. In a world moving away from classes for small units of logic, closures are how you encapsulate state.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`#`	Private field prefix — a `#` before a name inside a class marks it as truly private, reachable only from inside that class.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`binding`	Binding — the link between a name and the value it refers to.
`rest`	Rest — using `...` to collect several values into a single array or object.
`callback`	Callback — a function you hand to another function to be run later (e.g. when a click happens).
`module`	Module — a single code file with its own scope that shares code via `import` / `export` .

`makeCounter` returns after running, yet `count` lives on because the returned functions still close over it. That's encapsulation with no class needed.

```
// private state — `count` is reachable only through the returned functions
function makeCounter() {
  let count = 0;                        // captured by the closures below
  return {
    increment() { return ++count; },
    value()     { return count; },
  };
}
const c = makeCounter();
c.increment();   // 1
c.increment();   // 2
// `count` cannot be read or written from outside — true privacy

// factory: build a customized function
const multiplier = factor => n => n * factor;   // closes over `factor`
const triple = multiplier(3);
triple(10);      // 30
```

A practical cache (memoization)

The `cache` is hidden inside the closure — no other code can tamper with it. This pattern shows up in debouncing, throttling, and React hooks too.

```
function memoize(fn) {
  const cache = new Map();          // private, persists across calls
  return (arg) => {
    if (cache.has(arg)) return cache.get(arg);
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
}
const slowSquare = n => { /* expensive */ return n * n; };
const fastSquare = memoize(slowSquare);   // remembers past results
```

GOTCHA The historical `var`-in-a-loop bug is a closure issue: all callbacks created with `var i` share *one* `i` and see its final value. `let` gives each iteration its own binding, so the closures capture distinct values. (See Variables & scope.)

WHEN TO USE JavaScript has no built-in privacy for ordinary variables on objects — anything exposed can be read and overwritten from outside — and closures are the mechanism that gives you encapsulation anyway. The idea: a function permanently remembers the variables of the scope it was created in, even after that outer function has returned. So if you keep a value in an outer function and only expose inner functions that touch it, that value becomes genuinely private — reachable *only* through the doors you chose to leave open. **Why it mattered historically:** this was the standard way to model private state and modules for years before `class` and `#private` fields existed, and it's still the lightest tool for the job. It also quietly powers everything: every event handler, `setTimeout` callback, and React hook is a closure over the variables it references. **In practice:** use a closure for a small amount of private state with one or a few behaviors (counters, caches, debouncers, factories); prefer a class when you'll create many instances or need inheritance. Mental test: *one thing with hidden state* → closure; *a type you instantiate repeatedly* → class.

What closures enable

`private state` `returned fn over a local`

Data only the inner function(s) can access — encapsulation without classes.

`factory functions` `fn returning fn`

Generate specialized functions by closing over configuration (`multiplier(3)`).

`memoization / caching` `closed-over Map`

Persist results between calls in a hidden store.

`callbacks & handlers` `closes over context`

Every event handler / `setTimeout` callback closes over the variables it references — this is closures in daily use.

`partial application` `preset some args`

Fix some arguments now, supply the rest later — built on closures (or `.bind`).

Generators REFERENCE

IN PLAIN TERMS

> Most functions run start to finish in one go. A **generator** is a special function you can **pause and resume** — it hands you one value, freezes in place, and waits until you ask for the next one. Think of a vending machine dispensing items one at a time rather than dumping everything at once. This is useful for producing long (or even endless) sequences without building the whole thing in memory up front. It's an advanced tool you won't reach for daily, but it's good to understand what it is.

A generator (`function*`) is a function you can **pause and resume**. Each `yield` hands a value to the caller and freezes execution until they ask for the next one. That makes generators ideal for lazy sequences, custom iterables, and streaming through huge or infinite data without building it all in memory.

You won't write them every day, but they're worth recognizing: they power the iteration protocol (a generator is automatically iterable, so it works with `for…of` and spread), and libraries use them for things like ID sequences and paginated data streams.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`spread`	Spread — using `...` to expand a list or object into its individual pieces.
`argument`	Argument — the actual value you pass into a function when you call it.
`iterable`	Iterable — any value you can step through one item at a time with `for…of` (lists, text, and more).

`.next()` resumes until the next `yield`, returning `{ value, done }`. The function's local state survives between calls.

```
function* idMaker() {
  let id = 1;
  while (true) yield id++;   // infinite, but lazy — only computes on demand
}
const ids = idMaker();
ids.next().value;   // 1
ids.next().value;   // 2

// a generator is iterable — works with for...of and spread
function* range(start, end) {
  for (let i = start; i < end; i++) yield i;
}
[...range(0, 5)];               // [0, 1, 2, 3, 4]
for (const n of range(0, 3)) {} // 0, 1, 2

// yield* delegates to another iterable
function* all() { yield* range(0, 2); yield* ["a", "b"]; }
```

VERSION Generators are **ES2015**. **Async generators** (`async function*` with `for await…of`) are **ES2018** — used to stream chunks from a network or file source as they arrive.

Generator pieces

`function*` `declaration`

Defines a generator. Calling it returns a generator object; the body doesn't run until `.next()`.

`yield` `yield value`

Pauses and emits a value. Resumes (with the value passed to the next `.next(x)`) when asked.

`yield*` `yield* iterable`

Delegates to another generator/iterable, yielding all of its values.

`.next(v)` `method`

Resumes execution; returns `{ value, done }`. An argument becomes the result of the paused `yield`.

`.return() / .throw()` `methods`

Force the generator to finish, or inject an error at the pause point.

`async function*` `async generator`

Yields promises; consumed with `for await…of`. For streaming async data. **ES2018.**

async / await CORE

IN PLAIN TERMS

Some things take time: fetching data from the internet, reading a file. You don't want your whole page to freeze while waiting. `async` / `await` is the modern way to write "go do this slow thing, and continue once it's done" — but written so it reads as simply as ordinary top-to-bottom steps. `await` literally means "wait here for that slow result before moving on," without locking up everything else. This is one of the most-used features in real web apps, because almost every app talks to a server.

`async` / `await` is how modern JavaScript handles anything that takes time — network requests, file reads, timers — while keeping the code readable as if it were synchronous. An `async` function always returns a **promise**; `await` pauses *inside* it until a promise settles, then gives you the resolved value (or throws the rejection).

It doesn't block the page — under the hood it's still the promise/event-loop machinery (next part), just written linearly. The payoff is enormous: asynchronous data flows read top-to-bottom, and errors go through ordinary `try/catch`.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`promise`	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.
`fulfilled`	Fulfilled — a promise that has succeeded and now holds a value.
`rejected`	Rejected — a promise that has failed and now holds an error.
`module`	Module — a single code file with its own scope that shares code via `import` / `export`.

Inside an `async` function, `await` makes asynchronous calls read like synchronous steps — and `try/catch` catches rejections.

```
async function loadOrders(userId) {
  try {
    const res = await fetch(`/api/users/${userId}/orders`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const orders = await res.json();      // await unwraps the promise
    return orders;                         // becomes the resolved value
  } catch (err) {
    console.error("Failed to load orders:", err);
    return [];                             // graceful fallback
  }
}

// callers either await it (inside async) or use .then():
const orders = await loadOrders(42);
loadOrders(42).then(render);
```

Sequential vs parallel — a performance fork

If async calls don't depend on each other, start them together with `Promise.all` — awaiting them one by one is a common, invisible performance bug.

```
// ❌ sequential — each await waits for the previous (slow)
const user = await getUser(id);
const posts = await getPosts(id);     // didn't need to wait for user

// ✅ parallel — kick off together, await both
const [user2, posts2] = await Promise.all([getUser(id), getPosts(id)]);

// tolerate partial failure:
const results = await Promise.allSettled([a(), b(), c()]);
```

GOTCHA `await` inside `array.forEach()` does nothing useful — `forEach` won't wait. Use a `for…of` loop to await in sequence, or `Promise.all(arr.map(...))` to await in parallel. Also: **top-level** `await` works in ES modules (ES2022) but not in classic scripts or non-async functions.

VERSION `async` / `await` is **ES2017**. `Promise.allSettled` is **ES2020**, `Promise.any` is **ES2021**, top-level `await` is **ES2022**, and `Promise.withResolvers()` is **ES2024**.

async / await toolkit

`async function` `async () ⇒ {}`
Marks a function as asynchronous; it always returns a promise and may use `await` inside.

`await` `await promise`
Pauses the async function until the promise settles; yields the value or throws the rejection. Only inside `async` (or a module's top level).

`try / catch` `around await`
Catches rejected awaits exactly like synchronous errors — the main reason async/await reads so cleanly.

`Promise.all` `[…] → all values`
Runs promises in parallel; resolves with all results, or rejects on the **first** failure. The go-to for independent calls.

`Promise.allSettled` `[…] → statuses`
Waits for all and reports each as fulfilled/rejected — when partial success is acceptable. **ES2020.**

`Promise.race / any` `[…] → first`
`race` settles on the first to settle (win or fail); `any` resolves on the first *success*. Timeouts, fastest-source. **ES2021.**

`top-level await` `in modules`
Use `await` at a module's top level without wrapping it in an `async` function. **ES2022.**

Part 6 · Objects & Classes

Structured data and the two ways to model it: flexible object literals for plain data, and classes for types with behavior. Plus destructuring, the syntax that unpacks them cleanly.

Objects & object literals CORE

> IN PLAIN TERMS
>
> An **object** groups related information together under one name, using labels. Instead of three separate variables `userName`, `userAge`, `userEmail`, you keep one `user` object with `name`, `age`, and `email` inside it — like a contact card that holds several fields. Objects are the workhorse of JavaScript: almost all real data (a product, a blog post, a server's response) is shaped as an object. The **literal** part just means the handy `{ }` syntax for writing one out directly. This section covers building them, reading from them, and the everyday tools for working with them.

Objects are JavaScript's universal container — keyed collections of values that model everything from a form's fields to an API response to application config. The literal syntax `{ }` is so central that most data you touch *is* an object literal, and the modern shorthands make building them terse and expressive.

Keys are strings (or symbols); values are anything. You'll read and write properties with dot notation (`obj.name`) for known keys and brackets (`obj[key]`) for dynamic ones — the bracket form is what lets you look up a property by a variable.

▼ KEY TERMS IN THIS SECTION

`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`reference`	Reference — a pointer to an object; copying it copies the pointer, so two names can share one object.
`immutable`	Immutable — cannot be changed after it's created; you make a new value instead of altering the old one.
`mutate`	Mutate — to change a value in place (the opposite of immutable).
`shallow`	Shallow copy — a copy one level deep; nested objects inside are still shared with the original, not duplicated.
`spread`	Spread — using `...` to expand a list or object into its individual pieces.
`rest`	Rest — using `...` to collect several values into a single array or object.
`prototype`	Prototype — the object a JavaScript object inherits shared properties and methods from.
`getter`	Getter — a method that runs when you *read* a property, letting a value be computed on access.
`setter`	Setter — a method that runs when you *assign* to a property, letting you validate or react to writes.
`strict mode`	Strict mode — a stricter dialect that turns silent mistakes into clear errors; automatic in modules and classes.

*Property shorthand (`{ name }`), method shorthand (`greet() {}`), and computed keys (`[expr]: v`) are all **ES2015** and used constantly.*

```
const name = "Ada", age = 36;
const user = {
  name,                        // shorthand: { name: name }
  age,
  "full title": "Dr.",         // quoted key (has a space)
  greet() { return `Hi, ${this.name}`; },   // method shorthand
  [`id_${age}`]: true,         // computed key from an expression
};

user.name;          // "Ada"     dot notation (known key)
user["full title"]; // "Dr."     bracket notation (dynamic/odd keys)
const k = "age";
user[k];            // 36        look up by variable
```

Inspecting, copying & combining

`Object.entries()` + `for…of` / `map` is the standard way to iterate an object's key/value pairs. Spread is the standard way to copy-with-changes.

```
Object.keys(user);        // ["name", "age", ...]   own enumerable keys
Object.values(user);      // the values
Object.entries(user);     // [["name","Ada"], ...]  great for for...of / map

const patched = { ...user, age: 37 };       // spread copy + override (immutable update)
const merged  = Object.assign({}, a, b);     // mutate target with sources

"name" in user;           // true  — key exists?
user.hasOwnProperty("x"); // own (non-inherited) key?  (or Object.hasOwn(user,"x"))
delete user.age;          // remove a property
```

GOTCHA Objects are compared and assigned by **reference** — `{...obj}` and `Object.assign` make *shallow* copies, so nested objects stay shared. For deep independence use `structuredClone(obj)`. To make a config truly read-only, `Object.freeze(obj)` (shallow).

VERSION `Object.entries` / `values` are **ES2017**; object spread `{...}` is **ES2018**; `Object.fromEntries` is **ES2019**; `Object.hasOwn(obj, key)` (cleaner than `hasOwnProperty`) is **ES2022**; `Object.groupBy` is **ES2024**.

WHEN TO USE The plain object literal is JavaScript's default container because it's the most direct way to model a *thing with named fields* — a record, a config, a function's options. `Map` exists for a different job that objects do badly: when you're using the structure as a **dictionary** whose keys are dynamic, added and removed constantly, non-string, or order-sensitive. Objects coerce keys to strings, inherit keys from their prototype (so `"toString" in obj` can surprise you), and have no clean `.size` — `Map` fixes all of that and is optimized for frequent insertion/deletion. `Object.freeze` exists to make a value's immutability *enforced* rather than just intended, so an accidental write to a constant or config throws in strict mode instead of corrupting state silently. **In practice:** object literal for fixed, known-shape data (a *struct*); `Map` when you're treating it as a keyed *dictionary*; `freeze` to lock true constants.

Literal syntax & access

`property` `key: value`

A name/value pair. Keys are strings or symbols; values are anything.

`shorthand` `{ name }`

`{ name }` ≡ `{ name: name }` when a variable matches the key. **ES2015.**

`method` `name() {}`
Method shorthand inside a literal — a function-valued property with its own `this`.

`computed key` `[expr]: v`
Key computed from an expression at creation time. **ES2015.**

`getter / setter` `get x(){} / set x(v){}`
Accessor properties that run a function on read/write — `obj.x` looks like data but executes code.

`dot access` `obj.key`
Read/write a known, valid-identifier key.

`bracket access` `obj[expr]`
Read/write a key computed at runtime, or one that isn't a valid identifier.

`spread / rest` `{...obj}`
Copy/merge (spread) or collect remaining keys (rest). Shallow. **ES2018.**

Essential Object utilities

`Object.keys / values / entries` `(obj)`
Arrays of own enumerable keys / values / `[k,v]` pairs — the basis for iterating objects.

`Object.assign` `(target, ...src)`
Copies own enumerable props into `target` (mutates it). Shallow merge.

`Object.fromEntries` `(pairs)`
Builds an object from `[key, value]` pairs — the inverse of `entries`. **ES2019.**

`Object.freeze / isFrozen` `(obj)`
Makes an object immutable (shallow). Useful for constants/config.

`Object.hasOwn` `(obj, key)`
Safe own-key check; replaces `hasOwnProperty`. **ES2022.**

`Object.create / getPrototypeOf` `(…)`
Low-level prototype control — rarely needed directly now that classes exist.

`Object.defineProperty` `(obj, k, desc)`
Fine-grained property control (writable/enumerable/getters). For library/framework work.

Destructuring CORE

IN PLAIN TERMS

Destructuring is a shorthand for pulling values *out* of an object or list and into their own named variables, all in one tidy line. Instead of writing `const name = user.name` and `const email = user.email` separately, you write `const { name, email } = user` and get both at once. Think of it as unpacking a box and laying the contents out with labels in a single move. It's used constantly in modern code because it cuts down on repetitive typing and makes clear, up front, exactly which pieces of data you care about.

Destructuring pulls values *out* of objects and arrays into named variables in one line. It's everywhere in modern code: unpacking function parameters, reading fields off an API response, swapping variables, grabbing the first few items of an array. Once it clicks, it removes a huge amount of `const x = obj.x` boilerplate.

It pairs naturally with **defaults** (supply a value when the property is missing), **renaming** (bind to a different local name), and **rest** (collect the leftovers). The same syntax works in function parameters — which is exactly the options-object pattern from earlier.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`??`	Nullish coalescing operator — returns its right-hand value only when the left-hand one is `null` or `undefined` (a precise fallback).
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`rest`	Rest — using `...` to collect several values into a single array or object.
`parameter`	Parameter — a named input slot listed in a function's definition.
`argument`	Argument — the actual value you pass into a function when you call it.

Object destructuring

Renaming uses `key: newName`; defaults use `= value`; nesting mirrors the object's shape. The trailing `= {}` makes the whole argument optional.

```
const user = { name: "Ada", role: "admin", prefs: { theme: "dark" } };

const { name, role } = user;               // name="Ada", role="admin"
const { role: r, plan = "free" } = user;   // rename role→r, default plan
const { prefs: { theme } } = user;         // nested
const { name: n, ...rest } = user;         // rest collects the other keys

// in a parameter list (the options-object pattern):
function connect({ host = "localhost", port = 5432 } = {}) { /* ... */ }
```

Array destructuring

Array destructuring is positional (commas skip elements); object destructuring is by key. The `useState` pattern is array destructuring in disguise.

```
const [first, second] = scores;          // by position
const [head, ...tail] = list;            // head + the rest as an array
const [, , third] = row;                 // skip with holes
[a, b] = [b, a];                         // swap without a temp
const { data } = await fetch(url).then(r => r.json());   // grab one field

// React-style: array destructuring of a returned pair
const [value, setValue] = useState(0);
```

GOTCHA Destructuring a `null` or `undefined` **throws** (`Cannot destructure property … of null`). Guard with a default on the source: `const { x } = obj ?? {}`. And defaults fire only on `undefined`, never on `null`.

WHEN TO USE Destructuring exists to kill a specific kind of noise: the repetitive `const name = user.name; const role = user.role;` boilerplate, and the awkwardness of returning or receiving multiple values. Before it, pulling several fields off an object or the first items off an array meant a line each; destructuring names them all in one pattern that visually mirrors the data's shape. It's *why* APIs can cleanly return pairs — `const [value, setValue] = useState()` works because array destructuring unpacks the returned pair by position — and why function options read so well (`function f({ host, port })`). **Why people use it:** less ceremony, clearer intent, and it pairs with defaults and rest to handle missing or extra pieces gracefully. **In practice:** use it to remove repeated property access, to name the parts of a returned tuple, and in parameter lists — but don't force deeply nested patterns where a couple of plain accesses would read more clearly.

Destructuring forms

object `const {a, b} = obj`
Bind variables from matching keys. Order doesn't matter.

array `const [a, b] = arr`
Bind by position. Use commas to skip elements.

rename `{a: x}`
Bind property `a` to a local named `x`.

default `{a = 1} / [a = 1]`
Fallback when the value is `undefined`.

nested `{a: {b}}`
Reach into nested structures in one pattern.

rest `{a, ...rest} / [a, ...rest]`
Collect remaining properties/elements into a new object/array. Must be last.

param destructuring `function f({a, b})`
Unpack an argument directly in the parameter list — the options-object pattern.

swap `[a, b] = [b, a]`
Exchange two values without a temporary variable.

Classes CORE

> IN PLAIN TERMS
>
> A **class** is a blueprint for making many similar objects. If you're building a game with lots of enemies, you don't describe each one from scratch — you write one `Enemy` blueprint (what data each has, what each can do), then stamp out as many as you need from it. Each one made from the blueprint is called an *instance*. Classes bundle data together with the actions that work on that data, and let one blueprint build on another. They're a clean way to model "a kind of thing" you'll have many of.

Classes are a clean syntax for creating objects that bundle **data and behavior** — and for expressing *is-a* relationships through inheritance. Under the hood they're still JavaScript's prototype system, but you rarely need to think about that; the `class` syntax reads like other languages and covers the cases you'll actually hit: a `constructor`, methods, fields, `static` members, private `#fields`, getters/setters, and `extends`.

When to use a class vs a plain object/closure: reach for a class when you'll create **many instances of the same shape with shared behavior** (models, components, services), or when you want inheritance. For one-off bags of data, a plain object literal is simpler.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`#`	Private field prefix — a `#` before a name inside a class marks it as truly private, reachable only from inside that class.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`TDZ`	TDZ (temporal dead zone) — the period before a `let`/`const` is initialized, where using it throws an error.
`binding`	Binding — the link between a name and the value it refers to.
`callback`	Callback — a function you hand to another function to be run later (e.g. when a click happens).
`closure`	Closure — a function that remembers and keeps using variables from where it was created.
`prototype`	Prototype — the object a JavaScript object inherits shared properties and methods from.
`instance`	Instance — a specific object built from a class blueprint.
`constructor`	Constructor — the special method that sets up a new instance when you use `new`.
`static`	Static member — a property or method that belongs to the class itself, not to individual instances.
`getter`	Getter — a method that runs when you *read* a property, letting a value be computed on access.
`module`	Module — a single code file with its own scope that shares code via `import`/`export`.
`strict mode`	Strict mode — a stricter dialect that turns silent mistakes into clear errors; automatic in modules and classes.

Fields and methods declared in the body; `#name` is genuinely private (enforced by the language); `static` members belong to the class itself; getters are accessed without parentheses.

```
class Account {
  balance = 0;             // public field (instance property)
  #pin;                    // private field — only accessible inside the class
  static bankName = "Acme";// static: lives on the class, not instances

  constructor(owner, pin) {
    this.owner = owner;
    this.#pin = pin;
  }

  deposit(amount) {        // method (shared on the prototype)
    this.balance += amount;
    return this;           // return this → chainable
  }

  get summary() {          // getter — accessed as a property: acc.summary
    return `${this.owner}: $${this.balance}`;
  }

  #checkPin(p) { return p === this.#pin; }   // private method

  static open(owner) {     // static factory
    return new Account(owner, "0000");
  }
}

const acc = new Account("Ada", "1234");
acc.deposit(100).deposit(50);   // chaining
acc.summary;                    // "Ada: $150"   (getter, no parens)
Account.bankName;               // "Acme"        (static)
// acc.#pin                     // SyntaxError — truly private
```

Inheritance

`extends` sets up inheritance; `super(...)` calls the parent constructor (required before `this`); `super.method()` calls the parent's version.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  constructor(name) {
    super(name);             // must call super() before using `this`
    this.legs = 4;
  }
  speak() {                  // override
    return `${super.speak()} — woof!`;   // super.method() calls the parent
  }
}
new Dog("Rex").speak();      // "Rex makes a sound — woof!"
```

GOTCHA Passing a method as a callback loses `this` just like any function (`setTimeout(acc.deposit)` breaks). Fix it with a **class field arrow** (`deposit = () => {…}`) which binds `this` per instance, or `.bind(this)` in the constructor. (See *this and binding*.) Also: class bodies are always in strict mode, and class declarations are **not** hoisted (TDZ applies).

VERSION Classes are **ES2015**. Public/private **fields**, **private methods** (`#`), and `static` fields/blocks are **ES2022**. The `#x in obj` private-field *check* is also ES2022.

WHEN TO USE Classes exist to express the pattern of "a type you'll create many of, each bundling the same data with the same behavior" — and to make *inheritance* (one type specializing another) readable. JavaScript always had this capability through prototypes, but the raw prototype syntax was verbose and confusing, so `class` was added as a clear, familiar way to write it (it's the same prototype machinery underneath). The real value is bundling: a class keeps the data and the methods that operate on it in one place, and `#private` **fields** (enforced by the language) let you hide internal state so callers can't reach in and break invariants — the guarantee closures gave you, now first-class. **Why people choose it:** when you're writing `new` repeatedly and every method works on the same internal data, a class says so directly. **In practice:** class for many instances of one shape, shared behavior, or inheritance; a plain object literal for one-off structured data; a closure for a single stateful function. If you only ever make one instance, a module or object is usually simpler.

Class members

`constructor` `constructor(){}`
Runs on `new`. Initializes instance state. One per class; calls `super()` first in subclasses.

`method` `name(){}`
Shared behavior on the prototype — one copy for all instances.

`field` `x = value`
Per-instance property declared in the body. Initialized before the constructor body runs. **ES2022.**

`#private` `#x / #method()`
Truly private field/method — inaccessible outside the class body (language-enforced, not convention). **ES2022.**

`static` `static x / static m()`
Belongs to the class itself, not instances — for constants, factories, utilities. `static {}` blocks run once at class definition. **ES2022.**

`get / set` `get x(){} set x(v){}`
Accessors that look like properties but run code — for computed/validated values.

`extends` `class B extends A`
Inheritance — `B` instances get `A`'s members.

`super` `super() / super.m()`
Call the parent constructor / a parent method.

`new` `new C(args)`
Constructs an instance — allocates the object, runs the constructor with `this` bound to it.

`instanceof` `x instanceof C`
Tests class membership via the prototype chain (see Unary & relational).

Part 7 · Strings & Modules

Two syntax features you reach for constantly: template literals for building strings, and the module system for splitting code across files.

Template literals CORE

IN PLAIN TERMS

Template literals are a nicer way to build text, especially when you need to drop variables into the middle of a sentence. Older code glued strings together with `+` signs, which got messy fast. With template literals you write the sentence normally inside backticks (`` ` `` `) and mark the slots where values go with` ${} `– like a fill-in-the-blank:` `Hi ${name}, you have ${count} messages` `. They also let you write text across multiple lines easily. You'll use these constantly for labels, messages, and building web addresses.

Backtick strings (`` ` `` `…` `) are the modern way to build text. They **interpolate** expressions with ${…}`, span **multiple lines** without escape characters, and support **tagged templates** for advanced cases like safe HTML and styled-components. For nearly all string assembly — UI labels, URLs, log messages — they've replaced` +` concatenation.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`${ }`	Template placeholder — inside a backtick string, `${ }` marks a slot where a value or expression is inserted into the text.
`` ` ``	Backtick — the quote character that starts and ends a *template literal* (a string that allows `${ }` slots and multiple lines).
`expression`	Expression — any piece of code that produces a value, like `2 + 2` or `user.name`.
`concatenation`	Concatenation — joining pieces of text end to end into one string.
`interpolation`	Interpolation — inserting a value into the middle of a string (what `${ }` does).
`template`	Template literal — a string written in backticks that supports `${ }` value slots and multiple lines.

`${}` accepts any expression — ternaries, function calls, arithmetic — and coerces the result to a string.

```
const name = "Ada", count = 3;

// interpolation — any expression goes in ${ }
const msg = `Hi ${name}, you have ${count} ${count === 1 ? "item" : "items"}.`;

// multi-line, no \n needed
const html = `
  <li class="${active ? "on" : ""}">
    ${user.name}
  </li>`;

// build a URL safely
const url = `/api/users/${id}/orders?page=${page}`;
```

Tagged templates

A tag is a function called with the literal's pieces — the mechanism behind libraries like styled-components, `html…` and GraphQL's `gql…`. Niche to write, common to use.

```
// a tag function receives the string parts and the interpolated values
function highlight(strings, ...values) {
  return strings.reduce((out, str, i) =>
    out + str + (values[i] != null ? `<b>${values[i]}</b>` : ""), "");
}
const out = highlight`Hello ${name}, you have ${count} items`;
// "Hello <b>Ada</b>, you have <b>3</b> items"
```

GOTCHA Interpolation does **no escaping** — building HTML with `${userInput}` is an XSS risk if the value is untrusted. Use `textContent`, a templating library, or escape the value. For raw backticks/`${}` inside the text, escape with a backslash.

Template literal features

interpolation `${expr}`

Embeds the value of any expression, coerced to string. The everyday use.

multi-line `literal newlines`

Newlines inside backticks are preserved — no `\n` or string concatenation.

tagged template `tag…`

Calls `tag(strings, ...values)` to process the parts — for escaping, i18n, CSS-in-JS, DSLs.

String.raw `String.raw\n`

Built-in tag that returns the raw text with escapes uninterpreted — handy for regex/Windows paths. **ES2015.**

Modules CORE

> IN PLAIN TERMS
>
> As a program grows, you split it across many files to stay organized — but then those files need to share code with each other. **Modules** are the system for that: each file says what it wants to make available to others (**export**) and pulls in what it needs from elsewhere (**import**). Think of it like a kitchen where each station prepares certain things and passes them where they're needed, rather than everyone crammed into one pot. This is how every modern JavaScript project is structured.

Modules are how you split a program across files: each file declares what it makes available with `export`, and other files pull those in with `import`. ES Modules (ESM) are the standard everywhere now — browsers (`<script type="module">`), Node, and every bundler — and they're strict-mode and deferred by default.

The two export styles you'll mix: **named exports** (many per file, imported by exact name) for utilities and components, and a **default export** (one per file) for "the main thing" a module provides. Knowing when each applies keeps imports tidy.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`scope`	Scope — the region of code where a particular variable is visible and usable.
`binding`	Binding — the link between a name and the value it refers to.
`static`	Static member — a property or method that belongs to the class itself, not to individual instances.
`promise`	Promise — an object standing in for a value that isn't ready yet; it later succeeds or fails.
`tree-shake`	Tree-shaking — a build step that drops exported code nothing actually imports, shrinking the result.

Exporting

```
// named exports — as many as you like
export const API_URL = "https://api.example.com";
export function formatDate(d) { /* ... */ }
export class Client { /* ... */ }

// or export a list at the bottom
const helper = () => {};
export { helper, API_URL as BASE };   // rename on the way out

// default export — one per module ("the main thing")
export default function App() { /* ... */ }
```

Importing

Default imports name themselves; named imports must match the exported name (unless you `as`-rename). `import * as` gathers everything into one object.

```
import App from "./App.js";                  // default (you pick the name)
import { formatDate, API_URL } from "./util.js";  // named (exact names)
import { formatDate as fmt } from "./util.js";    // rename on import
import App, { API_URL } from "./App.js";   // default + named together
import * as util from "./util.js";         // everything as a namespace object
import "./styles.css";                     // side-effect only (no bindings)

// dynamic import — load on demand, returns a promise (code-splitting)
const { heavy } = await import("./heavy.js");
```

GOTCHA ESM imports are **static** — paths must be string literals (resolved before run), which is what lets bundlers tree-shake. For conditional/lazy loading use the **dynamic** `import()` function, which returns a promise. Browsers require real file extensions in paths (`./util.js`, not `./util`).

VERSION ES Modules are **ES2015** (and ubiquitous now). **Dynamic** `import()` is **ES2020**, as is `import.meta` (e.g. `import.meta.url`). **Import attributes** for JSON/other types — `import data from "./d.json" with { type: "json" }` — are **ES2025**; top-level `await` in modules is **ES2022**.

WHEN TO USE Modules exist to solve the original sin of browser JavaScript: everything shared one global namespace, so files stomped on each other's variables and load order became a fragile guessing game. `import`/`export` give each file its own scope and make dependencies *explicit* — you can see exactly what a file needs and what it offers, and tooling can follow those links to bundle, lazy-load, and **tree-shake** (drop unused exports). **Named vs default** are two intents: named exports suit a file that provides *several* things (a utilities module) and are explicit at the import site, so they refactor and tree-shake cleanly; a default export suits a file whose job is to provide *one* main thing (a component, a configured client). **Why people use them:** modules made large JavaScript codebases maintainable at all. **In practice:** named exports for multi-export files (many teams standardize on these everywhere for consistency); default for a module's single main export; be deliberate rather than mixing them arbitrarily.

Module syntax

`export (named)` `export const x`
Expose a binding by name. Many per file. Imported with `{ x }`.

`export list` `export { a, b as c }`
Export several at once, optionally renamed.

`export default` `export default v`
The single "main" export of a module. Imported without braces, under any name.

`re-export` `export … from "…"`
Forward another module's exports — used to build a barrel/index file.

`import (default)` `import X from "…"`
Bring in the default export; you choose the local name.

`import (named)` `import { a } from "…"`
Bring in named exports by exact name; `as` to rename.

`import * as ns` `namespace import`
Bind all named exports as properties of one object.

`import "…"` `side-effect import`

Run a module for its effects (CSS, polyfills) without binding anything.

`import()` `dynamic import`

Function form — loads a module at runtime, returns a promise. For code-splitting/lazy loading. **ES2020.**

`import.meta` `meta object`

Per-module metadata, e.g. `import.meta.url`. **ES2020.**

`import attributes` `with { type: "json" }`

Import non-JS resources like JSON with an explicit type. **ES2025.**

Part 8 · Iteration & Asynchrony

The two protocols that tie the language together: how anything becomes loopable, and how the engine runs your asynchronous code without ever blocking.

Iterators & iterables REFERENCE

> IN PLAIN TERMS
>
> You've seen that you can loop over a list with `for…of`. Ever wonder *why* that works on lists, text, and several other things but not just anything? The answer is a quiet agreement called the **iterable** protocol: any value that follows it promises "you can step through me one item at a time," and `for…of` (plus spread `...`) knows how to ask. Most of the time you just *use* iterables without thinking about the machinery. This section lifts the hood for the curious, and shows how to make your own thing loopable.

"Iterable" is the contract behind `for…of`, spread, and destructuring. Any object that implements `Symbol.iterator` can be looped, spread, and destructured — which is why arrays, strings, `Map`, `Set`, `NodeList`, and generators all work with the same syntax. You consume iterables constantly; you'll occasionally *make* one.

The newest addition, **Iterator Helpers** (ES2025), brings `map` / `filter` / `take` / `drop` / `reduce` directly to iterators — so you can process lazy or infinite sequences with array-like methods without materializing everything into an array first.

▼ KEY TERMS IN THIS SECTION

`===`	Strict equality operator — compares two values and returns `true` only if they're the same type and value, with no automatic conversion. Your default for comparing.
`==`	Loose equality operator — compares two values *after* converting them to a common type, which causes surprises. Avoid in favour of `===`.
`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`...`	Spread/rest operator (three dots) — either *spreads* a collection out into its items, or *gathers* loose items into one collection, depending on where it's used.
`++`	Increment operator — adds 1 to a number variable (`i++`).
`%`	Remainder (modulo) operator — gives what's left over after division (e.g. `7 % 3` is `1`); handy for wrapping and even/odd checks.
`syntax`	Syntax — the grammar rules for how code must be written so the computer can understand it.
`spread`	Spread — using `...` to expand a list or object into its individual pieces.
`destructuring`	Destructuring — unpacking values from an object or array into separate variables in one line.
`Symbol`	Symbol — a guaranteed-unique value, used mainly as a special, collision-proof object key.

Implementing `[Symbol.iterator]()` (easiest as a generator method) makes any object work with the whole iteration toolkit.

```
// these are all iterable → work with for...of, spread, destructuring
for (const ch of "hi") {}              // strings
for (const x of new Set([1, 2])) {}    // Sets
for (const [k, v] of new Map()) {}     // Maps → [key, value]
const arr = [...someSet];              // spread any iterable into an array

// make your own iterable (generators are the easy way):
const range = {
  *[Symbol.iterator]() { for (let i = 0; i < 3; i++) yield i; }
};
[...range];   // [0, 1, 2]
```

Iterator Helpers (ES2025)

These evaluate lazily — `take(3)` stops the pipeline after three results, so they work even on infinite generators.

```
// chainable, lazy methods on iterators — no intermediate arrays
const firstThree = numbers.values()
  .filter(n => n % 2 === 0)
  .map(n => n * 10)
  .take(3)
  .toArray();

Iterator.from(anyIterable);   // wrap an iterable as a helper-enabled iterator
```

VERSION The iteration protocol is **ES2015**. **Iterator Helpers** (the new `Iterator` global with `map` / `filter` / `take` / `drop` / `reduce` / `flatMap` / `some` / `find` / `every` / `toArray` and `Iterator.from`) are **ES2025** — check runtime support before relying on them in production.

The iteration protocol

`iterable` `[Symbol.iterator]()`

An object with a `Symbol.iterator` method returning an iterator. Makes it usable in `for…of`, spread, destructuring.

`iterator` `{ next() }`

An object whose `.next()` returns `{ value, done }`. Produced by the iterable's `Symbol.iterator`.

`for…of` `consumes iterables`

The everyday consumer of iterables (see Loops).

`spread / destructuring` `[...it]`

Also consume iterables — convert to arrays, unpack values.

`generators` `function*`

The simplest way to *produce* an iterator/iterable (see Generators).

`Iterator Helpers` `.map/.filter/.take…`

Lazy, chainable transforms on iterators. **ES2025.**

`async iterable` `[Symbol.asyncIterator]`

Consumed with `for await…of` — for streams of async data. **ES2018.**

Promises & the event loop CORE

IN PLAIN TERMS

When you order at a busy coffee counter, you don't stand frozen at the register until your drink is ready — you get a buzzer and step aside, and it goes off when the drink is done. A **promise** is that buzzer for code: when you ask for something that takes time (data from a server, a file, a timer), JavaScript hands you a promise *right away* that will later either succeed or fail. The **event loop** is the system that keeps the whole café running smoothly so the line never freezes while drinks are being made. This matters because JavaScript can only do one thing at a time, yet still has to feel responsive — and this is how it pulls that off.

A **promise** is an object representing a value that isn't ready yet — it's *pending*, then either *fulfilled* (with a value) or *rejected* (with an error). Promises are what `async` / `await` is built on; understanding them clears up why timing in JavaScript works the way it does.

JavaScript is **single-threaded**: one call stack, one thing at a time. The **event loop** is how it stays responsive anyway — slow work (timers, network, I/O) is handed off, and its callbacks are queued to run later when the stack is empty. Crucially, promise callbacks (*microtasks*) jump ahead of `setTimeout` callbacks (*macrotasks*), which explains a lot of "why did this log in that order?" puzzles.

▼ KEY TERMS IN THIS SECTION

`=>`	Arrow — defines an *arrow function*, a short way to write a function (e.g. `x => x * 2`). The arrow separates the inputs from what the function does.
`callback`	Callback — a function you hand to another function to be run later (e.g. when a click happens).

`.then` / `.catch` / `.finally` and `async` / `await` are two syntaxes for the same promises — mix per readability, but await usually wins.

```
// the three states, and the two ways to consume:
fetch("/api/data")              // returns a pending promise
  .then(res ⇒ res.json())       // chains: each .then returns a new promise
  .then(data ⇒ render(data))
  .catch(err ⇒ showError(err)) // handles any rejection in the chain
  .finally(() ⇒ hideSpinner());// runs regardless

// the same thing with async/await (usually clearer):
try {
  const res = await fetch("/api/data");
  render(await res.json());
} catch (err) { showError(err); }
finally { hideSpinner(); }
```

The event loop & ordering

Synchronous code finishes, then all microtasks (promise callbacks) run, then the next macrotask (timer). This ordering is the key to reasoning about async timing.

```
console.log("1");
setTimeout(() ⇒ console.log("4 (macrotask)"), 0);   // queued last
Promise.resolve().then(() ⇒ console.log("3 (microtask)")); // queued first
console.log("2");
// Order: 1, 2, 3, 4
// synchronous code runs first; microtasks (promises) drain
// before macrotasks (setTimeout) — even with a 0ms delay.
```

GOTCHA An **unhandled rejection** (a rejected promise with no `.catch` / `try`) is an error — attach a handler or `await` inside `try`. And `setTimeout(fn, 0)` doesn't run "immediately": it waits for the stack to clear *and* for all pending microtasks, so promises always win the race.

VERSION Promises are **ES2015**. `Promise.allSettled` **ES2020**, `Promise.any` **ES2021**, `Promise.withResolvers()` **ES2024**, and `Promise.try()` (run a function and capture sync throws or async rejections uniformly) **ES2025**.

WHEN TO USE Promises exist to tame asynchronous code. The old approach was callbacks, which nested into the infamous "pyramid of doom" — deeply indented, with error handling duplicated at every level and no way to compose steps. A promise is a first-class *object representing a future value*, so you can pass it around, chain steps with `.then`, and handle any failure in one `.catch`. `async` / `await` then exists to go one step further: it lets you *write* promise-based code as if it were synchronous — top to bottom, with ordinary `try/catch` for errors — while it still runs without blocking the single thread, because the event loop hands off the waiting and resumes you later. Understanding the loop (microtasks/promises run before macrotasks/timers) is *why* async ordering sometimes surprises people. **In practice:** prefer `async` / `await` for almost everything — it reads best; drop to `.then()` chains for a quick one-liner or a functional composition. Same promises underneath; choose by readability.

Promise toolkit

`states` `pending → fulfilled/rejected`

A promise settles once, to a value or an error, and never changes again.

`.then(onOk, onErr)` `method`

Registers success/failure callbacks; returns a new promise, enabling chains.

`.catch(onErr)` `method`

Handles a rejection anywhere earlier in the chain. Always end chains with one (or use try/catch).

`.finally(fn)` `method`

Runs on settle, success or failure — for cleanup. **ES2018.**

`new Promise` `(resolve, reject) ⇒ …`

Wrap a callback-based API into a promise. Most code consumes promises rather than constructing them.

`Promise.all / allSettled / race / any` `static`

Combine multiple promises — all-or-nothing, all-with-status, first-to-settle, first-success (see async/await).

`Promise.resolve / reject` `static`

Create an already-settled promise — useful in tests and adapters.

`microtask vs macrotask` `event loop`

Promise callbacks (microtasks) run before timers (macrotasks). The basis of async ordering. `queueMicrotask(fn)` schedules one directly.

Syntax Index

Every operator, keyword, and statement form in one place — the quick lookup for "what does this symbol mean?"

TOKEN	KIND	MEANING
`+`	Arithmetic / String	Addition, or string concatenation if either side is a string.
`-`	Arithmetic	Subtraction; unary negation.
`*`	Arithmetic	Multiplication.
`/`	Arithmetic	Division (`1/0` → `Infinity`, never throws).

TOKEN	KIND	MEANING
`%`	Arithmetic	Remainder — sign follows the dividend. Used for wrapping/even-odd.
`**`	Arithmetic	Exponentiation (`2 ** 10`). Right-associative. ES2016.
`++`	Arithmetic	Increment by 1 (prefix returns new, postfix old).
`--`	Arithmetic	Decrement by 1.
`=`	Assignment	Assign; itself an expression returning the value.
`+= -= *= /= %=`	Assignment	Compound arithmetic-assign (`x += 1`).
`**=`	Assignment	Exponentiate-assign. ES2016.
`&&=`	Assignment	Assign only if left is truthy. ES2021.
`||=`	Assignment	Assign only if left is falsy. ES2021.
`??=`	Assignment	Assign only if left is null/undefined — set defaults. ES2021.
`&= |= ^= <<= >>= >>>=`	Assignment	Compound bitwise-assign.
`===`	Comparison	Strict equality — no coercion. Your default.
`!==`	Comparison	Strict inequality.
`==`	Comparison	Loose equality — coerces. Avoid (except `x == null`).
`!=`	Comparison	Loose inequality. Avoid.
`> < >= <=`	Comparison	Relational; numeric for numbers, lexicographic for two strings.
`&&`	Logical	AND — returns left if falsy, else right. Short-circuits.
`||`	Logical	OR — returns left if truthy, else right. Fallback idiom.
`!`	Logical	NOT — boolean negation; `!!x` coerces to boolean.
`??`	Nullish	Coalescing — right side only if left is null/undefined. ES2020.
`?.`	Nullish	Optional chaining — safe access; yields undefined instead of throwing. ES2020.
`?.()  ?.[]`	Nullish	Optional call / optional dynamic access. ES2020.
`?:`	Conditional	Ternary — `cond ? a : b` ; the expression form of if/else.
`&`	Bitwise	AND — test/mask flags.
`|`	Bitwise	OR — combine flags.
`^`	Bitwise	XOR — toggle/parity.
`~`	Bitwise	NOT — invert bits.
`<< >> >>>`	Bitwise	Left / sign-propagating right / zero-fill right shift.
`typeof`	Unary	Returns a type-name string; safe on undeclared names.
`instanceof`	Relational	Prototype-chain membership test.
`in`	Relational	True if a key exists in an object (own or inherited).
`delete`	Unary	Remove a property from an object.
`void`	Unary	Evaluate an expression and yield undefined.
`new`	Operator	Construct an instance from a class/constructor.
`new.target`	Meta	Whether the current function was called with `new` .
`await`	Unary	Pause an async function until a promise settles.
`yield  yield*`	Operator	Pause/produce (and delegate) inside a generator.
`,`	Operator	Comma — evaluate both, return the last. Rare.

TOKEN	KIND	MEANING
`...`	Spread / Rest	Expand an iterable/object, or gather remaining items.
`const`	Declaration	Block-scoped, non-reassignable binding. Default choice.
`let`	Declaration	Block-scoped, reassignable binding.
`var`	Declaration	Legacy function-scoped, hoisted binding. Avoid.
`function`	Declaration	Function declaration — hoisted; has its own `this`.
`function*`	Declaration	Generator function — pausable via `yield`.
`async function`	Declaration	Async function — returns a promise; allows `await`.
`class`	Declaration	Class declaration — constructor, methods, fields, inheritance.
`=>`	Function	Arrow function — concise, lexical `this`, implicit return.
`if / else`	Statement	Conditional branching.
`switch / case / default`	Statement	Multi-way branch on `===`; mind `break`.
`for`	Statement	Counted loop with init/condition/step.
`for…of`	Statement	Iterate values of an iterable. Default value loop.
`for…in`	Statement	Iterate enumerable keys (objects; avoid on arrays).
`for await…of`	Statement	Iterate an async iterable/stream. ES2018.
`while / do…while`	Statement	Condition-driven loops (do…while runs once first).
`break / continue`	Statement	Exit a loop / skip to next iteration; support labels.
`return`	Statement	Return a value from a function (and exit it).
`throw`	Statement	Raise an error to the nearest catch.
`try / catch / finally`	Statement	Run risky code, handle errors, always clean up.
`import`	Module	Bring in exports (default, named, namespace, side-effect).
`import()`	Module	Dynamic import — returns a promise. ES2020.
`export`	Module	Expose bindings (named or default) from a module.
`import.meta`	Module	Per-module metadata (e.g. `import.meta.url`). ES2020.
`this`	Keyword	Call-site context; lexical in arrows.
`super`	Keyword	Call parent constructor/method in a subclass.
`true / false`	Literal	Boolean values.
`null`	Literal	Intentional empty value.
`undefined`	Literal	Absent/uninitialized value.
`NaN / Infinity`	Literal	Not-a-Number / numeric infinity.
`0n`	Literal	BigInt literal (n suffix). ES2020.
`` `…${}…` ``	Literal	Template literal — interpolation + multi-line.
`{ } / [ ]`	Literal	Object literal / array literal.
`// /* */`	Comment	Line / block comment.
`#!`	Directive	Hashbang — ignored first line for executable scripts. ES2023.
`"use strict"`	Directive	Opt into strict mode (implicit in modules/classes).